

IF2211: Laporan Tugas Besar 3

**Implementasi Pattern Matching pada Chromium Browser
Extension untuk Deteksi Judi Online**



Dipersiapkan oleh:

13524127

Fazri Arrashyi Putra

13524138

Ahmad Rinofaros Muchtar

13524147

Muh. Hartawan Haidir

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2025**

Daftar Isi

Deskripsi Tugas.....	1
Landasan Teori.....	4
2.1. Knuth-Morris-Pratt Algorithm.....	4
2.2. Boyer Moore Algorithm.....	4
2.3. Regular Expression.....	5
2.4. Weighted Levenshtein Distance Algorithm.....	5
2.5. Aho-Corasick Algorithm.....	5
Aplikasi KMP, Boyer-Moore, dan RegEx.....	6
3.1. Knuth-Morris-Pratt Algorithm.....	6
3.2. Boyer Moore Algorithm.....	8
3.3. Regular Expression.....	12
3.4. Weighted Levenshtein Distance Algorithm.....	13
3.7. Aho-Corasick Search.....	16
Implementasi dan Pengujian.....	19
4.1. Implementasi.....	19
4.2. Pengujian.....	54
4.3. Analisis Algoritma.....	55
Kesimpulan dan Saran.....	56
5.1. Kesimpulan.....	56
5.2. Saran.....	57
Lampiran.....	58
Tabel Penyelesaian.....	59
Tabel Pembagian Tugas.....	60
Daftar Pustaka.....	61

Bab 1

Deskripsi Tugas

Pada tugas besar ini, kami membuat sebuah chromium browser extension yang dapat mendeteksi informasi yang mengandung unsur judul. Sistem kami memiliki fitur dengan detail seperti berikut:

1. Extension yang dibuat pada tugas besar ini dikembangkan dengan bahasa Typescript. Dalam proses pencocokan kata kunci, sistem dapat mengimplementasikan algoritma Knuth-Morris-Pratt dan Boyer Moore serta RegEx. Untuk algoritma selain RegEx, pencarian dilakukan menggunakan file keyword.txt yang dipisahkan dengan newline untuk melakukan string matching secara iterative, sehingga pengguna tidak memasukkan input, tapi sistem yang membaca file tersebut secara langsung. Untuk pencarian dengan regex, format pencarian seminimal mungkin dapat handle 2 atau 3 angka di belakang kata tanpa spasi. Contoh: MAXWIN234, SLOT99. Test case yang diberikan dapat sangat bervariasi, sehingga pencarian RegEx dapat diimplementasikan dengan detail dan dapat handle banyak edge case.
2. Developer Judul juga membuat penamaan dari websitenya, seperti pada teknik Phishing, jadi membuat nama domain yang serupa dengan pada umumnya, namun nyatanya berbeda. Ini terjadi oleh karena adanya perbedaan kode komputer (unicode), dan komputer mengiranya adalah karakter yang berbeda. Maka dari itu, sistem juga bisa membaca kecocokan secara exact matching dan juga fuzzy matching (apabila tidak ditemukan kecocokan yang persis). Untuk setiap kata kunci yang tidak memiliki kemunculan sama sekali pada saat exact matching, lakukan pencarian kembali dengan perhitungan tingkat kemiripan menggunakan algoritma Weighted Levenshtein Distance. Penyesuaian bobot (weight) ini krusial karena pelaku sering kali menyiasati strategi dengan menggunakan huruf pengganti yang mirip secara visual (misalnya mengubah huruf 'O' menjadi angka '0', 'A' menjadi '4', atau 'I' menjadi '1'). Implementasi harus memperhatikan hal-hal berikut:

- a. Weighted Cost: Berikan bobot penalti yang lebih kecil untuk substitusi karakter yang mirip secara visual dibandingkan dengan karakter yang benar-benar berbeda.
 - b. Thresholding: Tentukan nilai ambang batas (threshold) kemiripan yang tepat untuk memastikan sistem tetap memberikan hasil yang relevan namun tetap menghindari munculnya false positive. Algoritma ini memungkinkan sistem untuk tetap menampilkan hasil pencarian yang relevan meskipun terdapat perbedaan minor, kesalahan ketik, atau upaya manipulasi teks oleh pengguna. Perlu diperhatikan bahwa secara konsep, exact matching juga merupakan bagian dari fuzzy matching.
3. Jadi sistem berjalan pengecekan dengan hirarki sebagai berikut:
 - a. Exact Matching: Algoritma pattern matching untuk mencari kata kunci dasar dari keyword.txt
 - b. RegEx Matching: Secara paralel atau sekuensial, jalankan Regex untuk menangkap pola <kata><angka>
 - c. Fuzzy Matching: Kalau string dalam DOM tidak lolos namun memiliki kemiripan, jalankan dengan Weighted Levenshtein Distance dalam upaya manipulasi karakter
4. Apabila ditemukan result keyword, maka elemen DOM terkait harus ditandai, dengan warna bebas. Namun highlight tidak boleh merusak layout, tepat sasaran dan dapat terhapus saat rescanning.
5. Saat user hover pada elemen yang terdeteksi, maka tampilkan tooltip dengan informasi berikut:
 - a. Keyword yang terdeteksi.
 - b. Algoritma yang digunakan.
 - c. Jumlah kemunculan.
 - d. Waktu eksekusiTooltip dibuat dengan custom DOM element.
6. Terdapat juga statistik secara realtime pada popup extension yang menampilkan:
 - a. Total keyword yang ditemukan

- b. Perbandingan jumlah keyword yang satu dengan lainnya, menggunakan visualisasi
- c. Waktu eksekusi tiap algoritma
- d. Jumlah match tiap algoritma

Bab 2

Landasan Teori

2.1. Knuth-Morris-Pratt Algorithm

Algoritma Knuth-Morris-Pratt merupakan sebuah string-searching algorithm yang mencari sebuah kemunculan word W dalam sebuah text string utama S dengan cara memperhatikan ketika sebuah ketidakcocokan terjadi, word W tersebut memberikan informasi yang cukup untuk menentukan dimana pencocokan berikutnya dapat dimulai sehingga bisa melewati pemeriksaan karakter dalam text string utama S yang sudah dicocokkan dengan word W

Algoritma ini menggunakan fungsi pembatas (border function) untuk menghitung urutan seberapa perbandingan harus dilakukan. Hal ini dilakukan dengan menghitung panjang prefix yang ada di sebuah string yang sama dengan suffix-nya.

Misal A adalah alfabet dan $x = x_1x_2\dots x_k$, $kA = \pi^2 N$, adalah string dengan panjang k yang terdiri atas karakter dalam alfabet A , maka prefix dari x adalah substring u dengan $u = x_1x_2\dots x_{k-1}$, $k \in \{1,2,\dots,k-1\}$ sehingga x diawali dengan u dan suffix dari x adalah substring v dengan $x_{k-b}x_{k-b+1}\dots x_k$, $k \in \{1,2,\dots,k-1\}$ sehingga x diakhiri v .

Border dari x adalah substring $r = x_1x_2\dots x_{k-1}$ dan $u = x_{k-b}x_{k-b+1}\dots x_k$, $k \in \{1,2,\dots,k-1\}$ sehingga r dan u adalah awalan dan akhiran sebenarnya dari x

2.2. Boyer Moore Algorithm

Algoritma Boyer Moore adalah salah satu algoritma yang paling efisien ketika digunakan dalam aplikasi umum. Algoritma ini bertugas untuk mencocokkan karakter dari sebelah kanan pattern. Dengan kata lain, algoritma ini akan memulai pencocokan karakter dari kanan, dan bukan dari kiri, maka akan lebih banyak informasi yang didapat. Kecepatan algoritma ini didukung oleh metode preprocessing dan aturan pergeseran (lompatan) yang cerdas :

1. Right-to-Left Scan : Berbeda dari algoritma biasa, pencocokan dilakukan dengan melihat karakter dari kanan ke kiri pada pola (pattern)

2. Preprocessing : Proses awal untuk menganalisis pattern sebelum pencarian dilakukan, yaitu menghasilkan dua aturan utama: *bad-character rule* dan *good-suffix rule*
3. Bad-Character Rule: Jika karakter pada teks tidak cocok dengan pattern, pola akan digeser ke kanan. Besarnya pergeseran didasarkan pada posisi kemunculan terakhir karakter yang tidak cocok di dalam pattern. Jika karakter tidak ada pattern, pola langsung di geser sepenuhnya melewati karakter tersebut
4. Good-Suffix Rule: Jika sebagian dari pattern telah cocok dengan teks namun kemudian terjadi ketidakcocokan pada karakter sebelumnya, pola akan digeser berdasarkan sub-pola (suffix) yang sudah cocok tersebut.

2.3. Regular Expression

Regular Expression (Regex) merupakan urutan karakter yang membentuk suatu search pattern. Pattern ini digunakan untuk pencocokan string, pencarian, dan manipulasi teks seperti menemukan, mengganti, atau mengekstrak data berdasarkan aturan tertentu.

Berbeda dengan algoritma string matching seperti KMP atau Boyer-Moore yang mencari string literal dengan persis, Regex memungkinkan pencarian pola yang dinamis dan fleksibel menggunakan metacharacters. Komponen Dasar Regex terdiri atas berikut:

1. Literal Character: Karakter biasa yang cocok dengan dirinya sendiri
2. Metacharacters : Simbol yang memiliki arti khusus
3. Escape Sequence: Simbol untuk menulis metacharacter sebagai karakter literal (/)

2.4. Weighted Levenshtein Distance Algorithm

Weighted Levenshtein Distance(WLD) adalah variasi dari Levenshtein Distance(LD) yang mengukur perbedaan antara dua sekuens string a dan b. Algoritma ini menghitung jumlah minimum biaya operasi untuk mengubah string a menjadi b. Berbeda dengan LD biasa, WLD dapat memberikan bobot yang berbeda pada setiap operasi sesuai kebutuhan domain

Misalkan $lev_w(a, b)$ adalah weighted edit distance antara string a dan b . Maka $lev_w(a, b)$ dapat didefinisikan secara rekursif sebagai:

$lev_w(i, j) = \text{biaya minimum untuk mengubah prefix } a[1..i] \text{ menjadi prefix } b[1..j].$

Dengan basis sebagai:

$$lev_w(0, j) = \sum (\text{biaya insert } b[k]) \text{ untuk } k=1..j$$

$$lev_w(i, 0) = \sum (\text{biaya delete } a[k]) \text{ untuk } k=1..i$$

Dengan rekursi

$$lev_w(i, j) = \min(\\ lev_w(i-1, j) + cost_delete(a[i]), \\ lev_w(i, j-1) + cost_insert(b[j]), \\ lev_w(i-1, j-1) + cost_substitute(a[i], b[j]) \\)$$

Dimana $lev_w(i-1, j-1) + cost_substitute(a[i], b[j]) = 0$ jika $a[i] == b[j]$

2.5. Aho-Corasick Algorithm

Algoritma ini merupakan multi-pattern string matching yang mampu mencari semua kemunculan dari sekumpulan kata kunci secara simultan dalam satu lintasan teks. Algoritma ini menggabungkan struktur data Trie dengan mekanisme failure link yang terinspirasi dari algoritma KMP. Berbeda dengan KMP dan Boyer-Moore yang hanya mencari satu pattern per eksekusi, Aho-Corasick membangun sebuah finite state automaton (FSA) dari seluruh daftar keyword sekaligus, sehingga kompleksitas pencariannya adalah $O(n + m + z)$.

2.6. Rabin-Karp Algorithm

Algoritma ini adalah algoritma string matching yang menggunakan teknik hashing untuk mendeteksi kemunculan pattern di dalam teks. Ide utamanya adalah membandingkan nilai hash dari pattern dengan nilai hash dari setiap substring teks yang panjangnya sama, sehingga perbandingan karakter satu persatu hanya dilakukan ketika nilai hash-nya cocok. Keunggulan rabin-karp terletak pada teknik rolling hash (hash bergeser). Daripada menghitung hash dari nol untuk setiap substring baru, algoritma ini memperbarui nilai hash secara inkremental dengan cara ;

- Menghapus kontribusi karakter paling kiri yang keluar dari jendela
- Menambahkan kontribusi karakter paling kanan yang baru masuk ke jendela

Rumus yang sering digunakan untuk rolling hash adalah

$$\text{hash}(s[i..i+m-1]) = (s[i] * \text{base}^{(m-1)} + s[i+1] * \text{base}^{(m-2)} + \dots + s[i+m-1]) \bmod \text{prime}$$

Pembaruan hash saat jendela bergeser satu langkah ke kanan:

$$\text{hash_baru} = (\text{hash_lama} - s[i] * \text{base}^{(m-1)}) * \text{base} + s[i+m]$$

Dengan pendekatan ini, pembaruan hash dilakukan dalam $O(1)$ per langkah, bukan $O(m)$. Akan tetapi, perlu diingat bahwa nilai hash dibatasi oleh modulus bilangan prima, dua string berbeda berpotensi menghasilkan hash yang sama (hash collision). Oleh karena itu, setiap kali nilai hash pattern cocok dengan hash substring, dilakukan verifikasi karakter per karakter untuk memastikan kecocokan yang sebenarnya. Jika verifikasi gagal, kejadian ini disebut spurious hit.

Bab 3

Aplikasi KMP, Boyer-Moore, dan RegEx

Bab ini menjelaskan kerangka berpikir implementasi masing-masing algoritma yang digunakan melalui pseudocode yang diekstrak langsung dari source code TypeScript, beserta penjelasan tiap bagiannya. Selain itu, terdapat langkah-langkah pemecahan masalah, proses pemetaan masalah menjadi elemen-elemen algoritma KMP dan BM, fitur fungsional dan arsitektur extension, serta contoh ilustrasi kasus

3.1. Knuth-Morris-Pratt Algorithm

a. Failure Function

Failure function merupakan inti dari algoritma KMP. Fungsi ini menghitung panjang prefix terpanjang dari sebuah pattern yang juga merupakan suffix-nya, sehingga ketika terjadi ketidakcocokan, algoritma dapat melanjutkan pencocokan pada posisi yang tepat tanpa mengulang dari awal.

```
FUNCTION failureFunction(pattern):  
  m <- panjang pattern  
  Failure[0..m-1] <- 0  
  j <- 0  
  i <- 1  
  
  WHILE i < m DO  
    IF pattern[i] == pattern[j] THEN  
      j <- j + 1  
      Failure[i] <- j  
      i <- i + 1  
    ELSE  
      IF j != 0 THEN  
        j <- failure[j - 1]
```

```

        ELSE
            Failure[i] <- 0
            i <= i + 1
RETURN failure

```

Dua pointer i (indeks saat ini) dan j (panjang prefix-suffix yang cocok) digunakan untuk mengisi tabel failure. Jika $\text{pattern}[i] = \text{pattern}[j]$ maka prefix sepanjang $j + 1$ telah ditemukan dan $\text{failure}[i] = j + 1$. Akan tetapi, jika tidak cocok dan $j \neq 0$ maka pointer j akan mundur ke $\text{failure}[j - 1]$ untuk mencoba prefix yang lebih pendek. Jika $j = 0$ dan tidak cocok, maka $\text{failure}[i] = 0$ dan i maju.

b. KMP Search

```

FUNCTION kmpSearch(text, pattern):
    n <- panjang text
    m <- panjang pattern
    indices <- []
    comparisons <- 0

    IF m == 0 OR m > n THEN RETURN { indices: [], ... }

    textUpper <- UPPERCASE(text)
    patternUpper <- UPPERCASE(pattern)
    failure <- failureFunction(patternUpper)

    i <- 0 // pointer pada text
    j <- 0 // pointer pada pattern

    WHILE i < n DO
        comparisons <- comparisons + 1

```

```

    IF textUpper[i] == patternUpper[j] THEN
        i <- i + 1
        j <- j + 1
        IF j == m THEN
            APPEND (i - m) TO indices      // match
ditemukan!
            j <- failure[j - 1]           // lanjut cari
match berikutnya
        ELSE
            IF j != 0 THEN
                j <- failure[j - 1]
            ELSE
                i <- i + 1

RETURN { indices, comparisons, executionTime }

```

Sebelum pencocokan, text maupun pattern dikonversi ke huruf kapital agar pencarian bersifat case-insensitive. Setelah itu, pointer i akan berjalan pada teks dan j pada pattern, keduanya maju bersama saat karakter cocok. Apabila $j = m$, berarti seluruh pattern telah ditemukan. Posisi awal match adalah $i - m$, dan j diperbarui ke $failure[j - 1]$ untuk melanjutkan pencarian tanpa mengulang dari awal. Ketika terjadi ketidakcocokan, jika $j > 0$ maka j mundur menggunakan tabel failure. Jika $j == 0$ maka pointer i yang maju

3.2. Boyer Moore Algorithm

a. Last Occurrence Table (Bad Character Rule)

Tabel kemunculan terakhir digunakan oleh aturan bad character pada Boyer-Moore. Tabel ini menyimpan indeks terakhir kemunculan setiap karakter di dalam pattern.

```
FUNCTION lastOccurrenceTable(pattern):  
    table <- Map (karakter -> indeks)  
  
    FOR i FROM 0 TO length(pattern) - 1 DO  
        table[pattern[i]] <- i      // setiap karakter  
        diperbarui ke indeks terbarunya  
  
    RETURN table
```

Tabel akan diisi dengan iterasi seluruh karakter pattern dari kiri ke kanan. Apabila ada satu karakter yang muncul lebih dari sekali, nilai yang disimpan adalah kemunculan paling kanan (terakhir). Tetapi, ketika terjadi bad character pada posisi j di pattern, algoritma menggeser pattern sehingga kemunculan terakhir karakter yang tidak cocok pada pattern sejajar dengan posisi tersebut di text.

b. Good Suffix Table

Good Suffix Rule digunakan ketika sebagian akhir pattern telah cocok dengan text, tetapi kemudian terjadi ketidakcocokan. Tabel ini akan menentukan seberapa jauh pattern dapat digeser berdasarkan suffix yang sudah cocok

```
FUNCTION goodSuffixTable(pattern):  
    m      <- panjang pattern  
    shift[0..m] <- m      // inisialisasi semua dengan m  
    border[0..m] <- 0
```

```

// Fase 1: hitung tabel border dari kanan ke kiri
i <- m; j <- m + 1
border[i] <- j

WHILE i > 0 DO
  WHILE j <= m AND pattern[i-1] != pattern[j-1] DO
    IF shift[j] == m THEN
      shift[j] <- j - i
      j <- border[j]
    i <- i - 1; j <- j - 1
  border[i] <- j

  // Fase 2: isi shift untuk prefix yang cocok
  sebagai suffix
  j <- border[0]
  FOR i FROM 0 TO m DO
    IF shift[i] == m THEN
      shift[i] <- j
    IF i == j THEN
      j <- border[j]

RETURN shift

```

Fase pertama adalah menghitung tabel border, yaitu informasi tentang prefix pattern yang sesuai dengan suffix-nya, lalu menetapkan nilai pergeseran awal. Setelah itu, fase kedua akan melengkapi nilai-nilai shift yang belum diisi pada fase pertama, menggunakan informasi prefix terpanjang yang juga merupakan suffix. Dan terakhir adalah nilai shift[j] menyatakan seberapa jauh pattern harus digeser ke kanan ketika terjadi ketidakcocokan.

c. Boyer-Moore Search

```
FUNCTION boyerMooreSearch(text, pattern):
    n <- panjang text
    m <- panjang pattern
    indices <- []
    comparisons <- 0

    IF m == 0 OR m > n THEN RETURN { indices: [], ... }

    textUpper <- UPPERCASE(text)
    patternUpper <- UPPERCASE(pattern)
    lastOcc <- lastOccurrenceTable(patternUpper)
    goodSuff <- goodSuffixTable(patternUpper)

    i <- 0 // posisi mulai pencocokan pada text

    WHILE i <= n - m DO
        j <- m - 1 // mulai dari karakter paling kanan
        pattern

        WHILE j >= 0 AND patternUpper[j] == textUpper[i +
j] DO
            comparisons <- comparisons + 1
            j <- j - 1

        IF j < 0 THEN
            // Seluruh pattern cocok
            APPEND i TO indices
            i <- i + goodSuff[0]
```

```

ELSE
    comparisons <- comparisons + 1
    lastPos      <- lastOcc[textUpper[i + j]] ??
-1
    badCharShift <- j - lastPos
    goodSuffShift <- goodSuff[j + 1]
    i <- i + MAX(badCharShift, goodSuffShift)

RETURN { indices, comparisons, executionTime }

```

Pencocokan dilakukan dari karakter yang paling kanan pattern (indeks $j = m - 1$), mundur ke kiri. Apabila $j < 0$ setelah loop dalam, berarti seluruh pattern cocok. Posisi 1 dicatat sebagai hasil, lalu pattern digeser sebesar `goodSuff[0]`. Apabila terjadi ketidakcocokan, akan dihitung sebagai dua nilai pergeseran, yaitu $\text{badCharShift} = j - \text{lastOcc}[\text{text}[i + j]]$ berdasarkan posisi terakhir karakter yang tidak cocok di dalam pattern dan kedua, yaitu $\text{goodSuffShift} = \text{goodSuff}[j + 1]$, berdasarkan suffix yang sudah cocok. Pattern akan digeser sebesar $\text{MAX}(\text{badCharShift}, \text{goodSuffShift})$ untuk memaksimalkan pergeseran. Selain itu, kompleksitas rata-rata yang didapatkan adalah $O(n/m)$

3.3. Regular Expression

Pencocokan Regular Expression dilakukan untuk menangkap pola kunci yang digabungkan langsung dengan angka, tanpa spasi pemisah, yang merupakan teknik lazim pada nama situs judi online.

```

FUNCTION regexSearch(text, keywords):
    results <- []

```

```

// Bangun pola: (keyword1|keyword2|...) diikuti 2-3
angka
pattern <- '(' + JOIN(keywords, '|') + ')([0-9]{2,3})'
  regex      <- COMPILE_REGEX(pattern, flags='gi') //
global, case-insensitive

WHILE match <- regex.exec(text) DO
  APPEND {
    fullMatch : match[0], // misal "SLOT99"
    keyword   : match[1], // misal "SLOT"
    digits    : match[2], // misal "99"
    index     : match.index // posisi di text
  } TO results

RETURN results

```

Pola Regular Expression dibangun secara dinamis dari daftar keyword yang telah tersimpan dari keyword.txt. Quantifier {2, 3} memastikan hanya 2 hingga 3 digit angka yang diterima, sehingga angka yang terlalu panjang (misalnya nomor telepon) tidak terkena false positive. Flag g memastikan semua kemunculan dalam teks ditemukan, bukan hanya pertama

3.4. Weighted Levenshtein Distance Algorithm

a. Weighted Levenshtein Core

```

FUNCTION getSubstitutionWeight(a, b):
  IF a == b THEN RETURN 0
  key <- a + b // misal "00" atau "I1"
  IF key IN VISUAL_SIMILARITY THEN

```

```

    RETURN VISUAL_SIMILARITY[key]
RETURN 1.0      // karakter berbeda, biaya penuh

FUNCTION weightedLevenshtein(a, b):
    m <- panjang a
    n <- panjang b
    dp[0..m][0..n] <- 0

    // Inisialisasi baris dan kolom pertama
    FOR i FROM 0 TO m DO dp[i][0] <- i
    FOR j FROM 0 TO n DO dp[0][j] <- j

    FOR i FROM 1 TO m DO
        FOR j FROM 1 TO n DO
            subWeight <-
getSubstitutionWeight(UPPER(a[i-1]), UPPER(b[j-1]))
            dp[i][j] <- MIN(
                dp[i-1][j] + 1,          // deletion
                dp[i][j-1] + 1,          // insertion
                dp[i-1][j-1] + subWeight // substitution
            (berbobot)
            )

    RETURN dp[m][n]

```

Matriks dp berukuran $(m + 1) \times (n + 1)$ inisialisasi: $dp[1][0] = 1$ dan $dp[0][1] = 1$ mewakili biaya menghapus atau menyisipkan i atau j karakter. Fungsi `getSubstitutionWeight` mengembalikan bobot penalti yang lebih kecil untuk pasangan karakter yang mirip secara visual (mis O -> 0 berbobot 0.1). Setiap

dp[i][j] diisi dengan nilai minimum dari tiga operasi edit. Kompleksitas waktu: $O(m \times n)$ kompleksitas ruang: $O(m \times n)$.

b. Fuzzy Matching dengan Thresholding

```
FUNCTION isFuzzyMatch(word, keyword, threshold =
1.5):
    IF |length(word) - length(keyword)| > 2 THEN
        RETURN false // optimasi awal, hindari
komputasi DP
        distance <- weightedLevenshtein(UPPER(word),
UPPER(keyword))
    RETURN distance <= threshold AND distance > 0

FUNCTION fuzzySearchAll(text, keywords, threshold =
1.5):
    results <- []
    words <- SPLIT(text, by=whitespace)
    currentIndex <- 0

    FOR EACH word IN words DO
        cleanWord <- REMOVE_NON_ALPHANUMERIC(word)

        IF length(cleanWord) >= 3 THEN
            FOR EACH keyword IN keywords DO
                IF isFuzzyMatch(cleanWord, keyword,
threshold) THEN
                    APPEND {
                        word : cleanWord,
                        keyword : keyword,
```

```

distance      :
weightedLevenshtein(UPPER(cleanWord),
UPPER(keyword)),
              index      : FIND_POSITION(word, text,
from=currentIndex)
              } TO results

currentIndex <- currentIndex + length(word) + 1

RETURN results

```

Teks akan dibagi menjadi kata-kata individual menggunakan whitespace sebagai pemisah. Setiap kata dibersihkan dari tanda baca dan karakter non-alphanumeric sebelum dicocokkan. Optimasi awal $|\text{len}(\text{word}) - \text{len}(\text{keyword})| > 2$ mengeliminasi pasangan kata yang panjangnya terlalu berbeda, menghindari komputasi DP yang tidak perlu. Kata dengan panjang kurang dari 3 karakter dilewati untuk mengurangi false positive. Hanya kata yang memenuhi kondisi $0 < \text{distance} \leq \text{threshold}$ yang dikembalikan sebagai hasil, sehingga exact match tidak tumpang tindih dan false positive diminimalkan

3.5. Aho-Corasick Search

```

FUNCTION ahoCorasickSearch(text, patterns):
  upperText <- UPPERCASE(text)
  results    <- Map: setiap pattern -> { indices: [],
comparisons: 0 }
  comparisons <- 0

  root <- buildTrie(patterns)

```

```

buildFailureLinks(root)

node <- root

FOR i FROM 0 TO length(upperText) - 1 DO
  ch <- upperText[i]

  // Ikuti failure link sampai ada transisi untuk ch,
atau sampai root
  WHILE node != root AND ch NOT IN node.children DO
    node <- node.fail
    comparisons <- comparisons + 1

  comparisons <- comparisons + 1

  IF ch IN node.children THEN
    node <- node.children[ch]
  ELSE
    CONTINUE // tetap di root, karakter tidak cocok
dengan pattern manapun

  // Laporkan semua pattern yang berakhir di posisi i
  IF node.output IS NOT EMPTY THEN
    FOR EACH matched IN node.output DO
      startPos <- i - length(matched) + 1
      results[matched].indices.APPEND(startPos)

RETURN results

```

Teks hanya diintasi satu kali (salah satu keunggulan utama aho-corasick dibandingkan KMP/BM berulang kali untuk setiap keyword. Loop WHILE mengikuti failure link

sampai ditemukan transisi yang valid untuk karakter saat ini. Ini ekuivalen dengan mekanisme “mundur” pada KMP. Posisi awal setiap match dihitung dari $i - \text{length}(\text{matched}) + 1$, karena i adalah indeks karakter terakhir dari match tersebut. Karena output sudah digabung dengan fail.output saat build, satu node dapat melaporkan beberapa pattern sekaligus (termasuk pattern yang merupakan suffix dari pattern lain).

3.6. Langkah-langkah Pemecahan Masalah

Sistem deteksi judi online pada browser extension ini dirancang dengan pendekatan berlapis. Langkah yang terjadi adalah :

1. **Pembacaan Keyword** sistem membaca daftar kata kunci dari file keyword.txt yang berisi kata kata terkait judi online, dipisahkan per baris. Kata kunci dinormalisasi ke huruf kecil saat dimuat dan dikonversi ke huruf kapital saat pencocokan dilakukan
2. **Ekstraksi Teks dari DOM** setelah halaman web selesai dimuat, sistem menggunakan TreeWalker untuk menelusuri node teks pada DOM. Node dari elemen script, style, iframe, canvas, dan elemen non-konten lainnya dilewati agar pencarian dilakukan pada konten yang terlihat pengguna
3. **Exact Matching** seluruh teks yang berhasil diekstrak digabung menjadi satu string dan dijalankan melalui algoritma KMP dan Boyer-Moore secara berurutan. Setiap keyword dalam daftar dicocokkan satu per satu terhadap teks tersebut
4. **Regex Matching secara paralel**, regular expression dijalankan untuk menangkap pola <kata><angka> yang tidak tertangkap oleh exact matching
5. **Fuzzy Matching** untuk setiap keyword yang tidak ditemukan sama sekali pada tahap exact matching, sistem menjalankan Weighted Levenshtein Distance terhadap setiap kata dalam teks. Tahap ini menangkap obfuskasi karakter seperti GAC0R, SL0t, dan H0K1
6. **Highlighting dan Tooltip Setiap posisi** yang terdeteksi dipetakan kembali ke node teks DOM asalnya. Teks yang cocok dibungkus dalam elemen dengan class judol-highlight dan diwarnai. Saat pengguna hover, tooltip menampilkan informasi detail deteksi.

7. **Statistik Realtime** Hasil dari seluruh tahap dikumpulkan dan dikirim ke popup extension untuk ditampilkan sebagai statistik: total keyword ditemukan, perbandingan antar algoritma, waktu eksekusi, dan jumlah match per algoritma.

3.7. Pemetaan Masalah ke Elemen algoritma KMP dan Boyer-Moore

Masalah deteksi konten judi online pada dasarnya adalah masalah string matching: diberikan sebuah teks panjang dan sekumpulan pattern, maka harus ditemukan semua posisi kemunculan setiap pattern di dalam teks tersebut. Berikut adalah pemetaan masalah ke elemen algoritma

Elemen Masalah	Elemen Algoritma
Konten halaman web (teks DOM)	Text string tempat pencarian dilakukan
Kata kunci dari keywords.txt	Pattern string yang dicari
Satu keyword dicari di seluruh teks	Satu pemanggilan kmpSearch() atau boyerMooreSearch()
Banyak keyword dicari sekaligus	Loop iteratif pada kmpSearchMultiple() / bmSearchMultiple()
Posisi kemunculan keyword	Nilai indeks yang dikembalikan dalam array indices
Variasi huruf besar/kecil	Normalisasi toUpperCase() pada text dan pattern sebelum pencocokan
Pengulangan pencarian tanpa mengulang dari awal	Failure Function (KMP) / Failure Link (Aho-Corasick)
Lompatan cerdas saat tidak cocok	Bad Character Rule + Good Suffix Rule (Boyer-Moore)
Mengukur efisiensi algoritma	Penghitungan comparisons dan executionTime pada setiap hasil

3.8. Fitur fungsional dan Arsitektur extension

1. Fitur Fungsional

Extension ini memiliki fitur fungsional sebagai berikut :

1. Exact Matching: Mencari keyword judi secara tepat menggunakan KMP dan Boyer-Moore. Pencarian bersifat case-insensitive. Keyword dibaca dari keywords.txt secara otomatis tanpa input pengguna.
2. Regex Matching: Mendeteksi pola <kata><angka> seperti SLOT99, MAXWIN234, TOGEL4D menggunakan Regular Expression yang dibangun secara dinamis dari daftar keyword.
3. Fuzzy Matching: Mendeteksi obfuskasi karakter menggunakan Weighted Levenshtein Distance. Karakter pengganti yang mirip secara visual (O→0, I→1, A→4, dst.) mendapat bobot penalti lebih kecil sehingga tetap terdeteksi.
4. DOM Highlighting: Elemen teks yang terdeteksi dibungkus dengan latar merah transparan. Highlight tidak merusak layout dan dihapus otomatis saat rescane.
5. Tooltip Informatif: Saat pengguna hover pada teks yang di-highlight, muncul tooltip yang menampilkan: keyword yang terdeteksi, teks yang cocok (matchText), algoritma yang digunakan, jumlah kemunculan, waktu eksekusi, dan jumlah perbandingan karakter.
6. Popup Statistik Realtime: Popup extension menampilkan: total keyword yang ditemukan, bar chart perbandingan jumlah match per algoritma, waktu eksekusi tiap algoritma, dan jumlah match tiap algoritma.
7. Blur/Censor (Bonus): Konten yang terdeteksi dapat diburamkan (blur) secara otomatis. Pengguna dapat mengklik area yang diblur untuk menampilkan kontennya kembali.
8. OCR pada Gambar (Bonus): Sistem menggunakan Tesseract.js untuk mengekstrak teks dari gambar di halaman web, kemudian menjalankan pencocokan keyword pada teks hasil OCR tersebut.

9. Aho-Corasick (Bonus): Sebagai alternatif multi-pattern matching yang lebih efisien, algoritma Aho-Corasick dapat diaktifkan dari popup untuk mencari semua keyword dalam satu lintasan teks.

3.9. Contoh Ilustrasi Kasus

a. Kasus 1 – Exact match Langsung

Teks: "Daftar sekarang di situs GACOR terpercaya"

Keyword: GACOR

- KMP menemukan match di indeks 26, menggunakan failure function untuk tidak mengulang dari awal saat ketidakcocokan.
- Boyer-Moore menemukan match di indeks 26, melompat menggunakan bad character rule sehingga jumlah perbandingan lebih sedikit.
- Hasil: Kata "GACOR" di-highlight dengan warna merah transparan. Tooltip menampilkan algoritma yang digunakan beserta waktu eksekusi.

b. Regex Match (Keyword + angka)

Teks: "Main di SLOT88 dan raih MAXWIN234 setiap hari"

Keyword di keywords.txt: SLOT, MAXWIN

- Tahap exact matching tidak menemukan SLOT atau MAXWIN secara berdiri sendiri (karena keduanya menyatu dengan angka).
- Tahap Regex dengan pola `/(SLOT|MAXWIN)([0-9]{2,3})/gi` menemukan SLOT88 di indeks 8 dan MAXWIN234 di indeks 23.
- Hasil: Kedua kata di-highlight, tooltip menampilkan algoritma "regex".

c. Fuzzy Match (obfuskasi Karakter)

Teks: "Bergabunglah di situs GACOR terbaik"

Keyword: GACOR

- Tahap exact matching tidak menemukan GACOR (karena 'O' diganti '0').
- Tahap Regex tidak cocok (tidak ada angka di belakang).
- Tahap Fuzzy Matching menghitung `weightedLevenshtein("GAC0R", "GACOR") = 0.1` (substitusi $O \rightarrow 0$ berbobot 0.1), yang berada di bawah threshold 1.5.

- Hasil: GACOR terdeteksi sebagai fuzzy match dari keyword GACOR dan di-highlight. Tooltip menampilkan algoritma "fuzzy" dan jarak Levenshtein.

d. Multiple Keyword Sekaligus (Aho-Corasick)

Teks: "Situs SLOT TOGEL GACOR MAXWIN terpercaya 2024"

Keywords: SLOT, TOGEL, GACOR, MAXWIN

- Dengan KMP/BM, pencarian dilakukan 4 kali (satu per keyword), total $4 \times O(n)$.
- Dengan Aho-Corasick, seluruh keyword dibangun ke dalam satu automaton dan teks hanya dilintasi satu kali, menemukan semua 4 keyword sekaligus dalam $O(n + \Sigma m)$.
- Hasil: Keempat keyword ditemukan dan di-highlight, popup menampilkan 4 total match dengan algoritma "aho-corasick".

e. Teks Bersi (Tidak ada Deteksi)

Teks: "Selamat datang di portal berita olahraga nasional"

Keywords: SLOT, GACOR, TOGEL, MAXWIN

- Tahap exact matching: tidak ada match.
- Tahap regex: tidak ada pola <kata><angka> yang cocok.
- Tahap fuzzy: semua jarak Levenshtein terhadap setiap kata di teks melebihi threshold 1.5.
- Hasil: Tidak ada highlight. Popup menampilkan total 0 keyword ditemukan. Sistem tidak memberikan false positive pada teks yang tidak mengandung konten judul.

Bab 4

Implementasi dan Pengujian

4.1. Implementasi

a. Arsitektur Sistem

i. Algorithm

a. kmp.ts

```
export interface MatchResult {
  indices: number[];
  executionTime: number;
  comparisons: number;
}

export function failureFunction (pattern:
string): number[] {
  const m = pattern.length;
  const failure: number[] = new
Array(m).fill(0);

  let j = 0;
  let i = 1;

  while (i < m) {
    if (pattern[i] === pattern[j]) {
      j++;
      failure[i] = j;
      i++;
    } else {
      if (j !== 0) {
        j = failure[j - 1];
      } else {
        failure[i] = 0;
        i++;
      }
    }
  }

  return failure;
}
```

```

}

export function kmpSearch (text: string, pattern:
string): MatchResult {
    const startTime = performance.now();
    const indices: number[] = [];
    let comparisons = 0;

    const n = text.length;
    const m = pattern.length;

    if (m === 0 || m > n) {
        return {
            indices: [],
            executionTime: performance.now() -
startTime,
            comparisons: 0,
        };
    }

    const textUpper = text.toUpperCase();
    const patternUpper = pattern.toUpperCase();
    const failure =
failureFunction(patternUpper);

    let i = 0;
    let j = 0;

    while (i < n) {
        comparisons++;

        if (textUpper[i] === patternUpper[j]) {
            i++;
            j++;

            if (j === m) {
                indices.push(i - m);
                j = failure[j - 1];
            }

```

```

        } else {
            if (j !== 0) {
                j = failure[j - 1];
            } else {
                i++;
            }
        }
    }
    return {
        indices,
        executionTime: performance.now() -
startTime,
        comparisons,
    };
}

export function kmpSearchMultiple (
    text: string,
    keywords: string[]
): Map<string, MatchResult> {
    const results = new Map<string, MatchResult>();

    for (const keyword of keywords) {
        const result = kmpSearch(text, keyword);
        if (result.indices.length > 0) {
            results.set(keyword, result);
        }
    }

    return results;
}

```

b. boyer-moore.ts

```

import { MatchResult } from "../kmp";

export function lastOccurrenceTable (pattern: string):
Map<string, number> {

```

```

    const table = new Map<string, number>();

    for (let i = 0; i < pattern.length; i++) {
        table.set(pattern[i], i);
    }

    return table;
}

export function goodSuffixTable(pattern: string):
number[] {
    const m = pattern.length;
    const shift: number[] = new Array(m + 1).fill(m);
    const border: number[] = new Array(m + 1).fill(0);

    let i = m;
    let j = m + 1;
    border[i] = j;

    while (i > 0) {
        while (j <= m && pattern[i - 1] !== pattern[j -
1]) {

            if (shift[j] === m) {
                shift[j] = j - i;
            }

            j = border[j];
        }
        i--;
        j--;
        border[i] = j;
    }
    j = border[0];
    for (i = 0; i <= m; i++) {
        if (shift[i] === m) {
            shift[i] = j;
        }

        if (i === j) {
            j = border[j];
        }
    }
}

```

```

        return shift;
    }

export function boyerMooreSearch (text: string,
pattern: string): MatchResult {
    const startTime = performance.now();
    const indices: number[] = [];
    let comparisons = 0;

    const n = text.length;
    const m = pattern.length;

    if (m === 0 || m > n) {
        return {
            indices: [],
            executionTime: performance.now() -
startTime,
            comparisons: 0,
        };
    }
    const textUpper = text.toUpperCase();
    const patternUpper = pattern.toUpperCase();
    const lastOccurrence =
lastOccurrenceTable(patternUpper);
    const goodSuffix = goodSuffixTable(patternUpper);

    let i = 0;
    while (i <= n - m) {
        let j = m - 1;

        while (j >= 0 && patternUpper[j] ===
textUpper[i + j]) {
            comparisons++;
            j--;
        }

        if (j < 0) {
            indices.push(i);
            i += goodSuffix[0];
        }
    }
}

```

```

        } else {
            comparisons++;

            const lastPos =
lastOccurrence.get(textUpper[i + j]) ?? -1;
            const badCharShift = j - lastPos;
            const goodSuffixShift = goodSuffix[j + 1];

            i += Math.max(badCharShift,
goodSuffixShift);
        }
    }
    return {
        indices,
        executionTime: performance.now() - startTime,
        comparisons,
    };
}

export function bmSearchMultiple(
    text: string,
    keywords: string[]
): Map<string, MatchResult> {
    const results = new Map<string, MatchResult>();

    for (const keyword of keywords) {
        const result = boyerMooreSearch(text, keyword);
        if (result.indices.length > 0) {
            results.set(keyword, result);
        }
    }

    return results;
}

```

c. regex.ts

```

export interface RegexMatchResult {
    match: string;
}

```

```

    index: number;
    algorithm: 'regex';
    executionTime: number;
}

const JUDOL_WORDS = [
    'slot', 'gacor', 'maxwin', 'hoki', 'madu', 'petir',
    'zeus',
    'dewa', 'jp', 'cuan', 'bonanza', 'wild', 'jackpot',
    'scatter',
    'spin', 'rtp', 'pragmatic', 'habanero', 'judi',
    'togel',
    'casino', 'taruhan', 'betting', 'megaways',
    'starlight',
    'gates', 'olympus', 'sultan',
];

const IMPERIAL_WORDS = [
    'BET', 'WIN', 'LUCKY', 'VIP', 'BONUS', 'FREE',
    'BIG', 'MEGA',
    'SUPER', 'ULTRA', 'HOT', 'COOL', 'BEST', 'TOP',
    'JACKPOT',
    'JACK', 'POT', 'PAY', 'GIFT', 'CASH',
];

const JUDOL_WORDS_JOIN = JUDOL_WORDS.join('|');
const IMPERIAL_WORDS_JOIN = IMPERIAL_WORDS.join('|');

const WORD_PATTERNS: RegExp[] = [
    new
    RegExp(`\\b(?:${JUDOL_WORDS_JOIN})\\s*\\d{2,3}\\b`,
    'gi'),
    new
    RegExp(`\\b\\d{2,3}\\s*(?:${JUDOL_WORDS_JOIN})\\b`,
    'gi'),
    new RegExp(`\\b(?:${JUDOL_WORDS_JOIN})\\d{2,3}\\b`,
    'gi'),
    new RegExp(`\\b\\d{2,3}(?:${JUDOL_WORDS_JOIN})\\b`,
    'gi'),

```

```

        new
RegExp(`\\b(?:${IMPERIAL_WORDS_JOIN})\\s*\\d{2,3}\\b`,
'gi'),
        new
RegExp(`\\b\\d{2,3}\\s*(?:${IMPERIAL_WORDS_JOIN})\\b`,
'gi'),
        new
RegExp(`\\b(?:${IMPERIAL_WORDS_JOIN})\\d{2,3}\\b`,
'gi'),
        new
RegExp(`\\b\\d{2,3}(?:${IMPERIAL_WORDS_JOIN})\\b`,
'gi'),
        new RegExp(`\\b(?:${JUDOL_WORDS_JOIN})\\b`, 'gi'),
];

export function regexSearchAll(text: string,
executionLimit: number = 2000): RegExpMatchResult[] {
    const startTime = performance.now();
    const seen = new Set<string>();
    const results: RegExpMatchResult[] = [];
    let matchCount = 0;

    for (const pattern of WORD_PATTERNS) {
        const regex = new RegExp(pattern.source,
pattern.flags);
        let m: RegExpExecArray | null;

        while ((m = regex.exec(text)) !== null) {
            if (matchCount >= executionLimit) break;
            const key = m.index + ':' + m[0];
            if (!seen.has(key)) {
                seen.add(key);
                results.push({
                    match: m[0],
                    index: m.index,
                    algorithm: 'regex',
                    executionTime: 0,
                });
                matchCount++;
            }
        }
    }
}

```

```

        }
        if (m.index === regex.lastIndex)
regex.lastIndex++;
    }
    if (matchCount >= executionLimit) break;
}

const totalTime = performance.now() - startTime;
for (const r of results) {
    r.executionTime = totalTime;
}

return results;
}

```

d. levenshtein.ts

```

const VISUAL_SIMILARITY: Map<string, number> = new
Map([
    ["O0", 0.1], ["0O", 0.1],
    ["I1", 0.1], ["1I", 0.1],
    ["L1", 0.2], ["1L", 0.2],
    ["A4", 0.2], ["4A", 0.2],
    ["E3", 0.2], ["3E", 0.2],
    ["S5", 0.2], ["5S", 0.2],
    ["B8", 0.2], ["8B", 0.2],
    ["T7", 0.3], ["7T", 0.3],
    ["G9", 0.3], ["9G", 0.3],
    ["G6", 0.3], ["6G", 0.3],
    ["0@", 0.2], ["@0", 0.2],
    ["S$", 0.1], ["$S", 0.1],
]);

function getSubstitutionWeight(a: string, b: string):
number {
    if (a === b) return 0;
    const key = a + b;
    return VISUAL_SIMILARITY.get(key) ?? 1.0;
}

```

```

}

export function weightedLevenshtein(a: string, b:
string): number {
    const m = a.length;
    const n = b.length;

    const dp: number[][] = Array.from(
        { length: m + 1 },
        () => new Array(n + 1).fill(0)
    );

    for (let i = 1; i <= m; i++) dp[i][0] = dp[i -
1][0] + 1;
    for (let j = 1; j <= n; j++) dp[0][j] = dp[0][j -
1] + 1;

    for (let i = 1; i <= m; i++) {
        for (let j = 1; j <= n; j++) {
            const subWeight = getSubstitutionWeight(
                a[i - 1].toUpperCase(),
                b[j - 1].toUpperCase()
            );
            dp[i][j] = Math.min(
                dp[i - 1][j] + 1,
                dp[i][j - 1] + 1,
                dp[i - 1][j - 1] + subWeight
            );
        }
    }
    return dp[m][n];
}

export function isFuzzyMatch(
    word: string,
    keyword: string,
    threshold: number = 1.5
): boolean {
    if (Math.abs(word.length - keyword.length) > 2)

```

```

return false;

    const distance = weightedLevenshtein(
        word.toUpperCase(),
        keyword.toUpperCase()
    );
    return distance <= threshold && distance > 0;
}

export interface FuzzyMatchResult {
    word: string;
    keyword: string;
    distance: number;
    startIndex: number;
}

export function fuzzySearchAll(
    text: string,
    keywords: string[],
    threshold: number = 1.5
): FuzzyMatchResult[] {
    const results: FuzzyMatchResult[] = [];
    const n = text.length;
    let i = 0;

    while (i < n) {
        if (/\\s/.test(text[i]) ||
            /[a-zA-Z0-9]/.test(text[i])) {
            i++;
            continue;
        }

        let j = i;
        while (j < n && /[a-zA-Z0-9]/.test(text[j])) {
            j++;
        }

        const word = text.substring(i, j);
        if (word.length >= 3) {
            for (const keyword of keywords) {

```

```

        if (isFuzzyMatch(word, keyword,
threshold)) {
            results.push({
                word,
                keyword,
                distance: weightedLevenshtein(
                    word.toUpperCase(),
                    keyword.toUpperCase()
                ),
                startIndex: i,
            });
        }
    }

    i = j;
}

return results;
}

```

e. normalize.ts

```

const HOMOGLYPH_MAP: Map<string, string> = new Map([
    ['α', 'a'], ['a', 'a'], ['à', 'a'], ['á', 'a'],
    ['â', 'a'],
    ['ã', 'a'], ['ä', 'a'], ['å', 'a'], ['æ', 'a'],
    ['e', 'e'], ['è', 'e'], ['é', 'e'], ['ê', 'e'],
    ['ë', 'e'],
    ['i', 'i'], ['ì', 'i'], ['í', 'i'], ['î', 'i'],
    ['ï', 'i'],
    ['o', 'o'], ['ò', 'o'], ['ó', 'o'],
    ['ô', 'o'],
    ['õ', 'o'], ['ö', 'o'], ['ø', 'o'],
    ['u', 'u'], ['ù', 'u'], ['ú', 'u'], ['û', 'u'],
    ['ü', 'u'],
    ['ç', 'c'], ['ç', 'c'],
    ['B', 'b'],

```

```

    ['p', 'p'], ['h', 'h'], ['x', 'x'], ['k', 'k'],
    ['m', 'm'],
    ['v', 'v'], ['n', 'n'],
    ['0', '0'], ['1', '1'], ['2', '2'], ['3', '3'],
    ['4', '4'], ['5', '5'], ['6', '6'], ['7', '7'],
    ['8', '8'], ['9', '9'],
  ]));

export function normalizeText(text: string): string {
  let result = '';
  for (const ch of text) {
    const mapped =
HOMOGLYPH_MAP.get(ch.toLowerCase());
    result += mapped ?? ch;
  }
  return result;
}

export const HOMOGLYPH_KEYS =
Array.from(HOMOGLYPH_MAP.keys());

```

f. pipeline.ts

```

import { kmpSearchMultiple, MatchResult } from './kmp';
import { bmSearchMultiple } from './boyer-moore';
import { regexSearchAll } from './regex';
import { fuzzySearchAll } from './levenshtein';
import { ahoCorasickSearch } from './aho-corasick';
import { rabinKarpSearchMultiple } from './rabin-karp';
import { normalizeText } from './normalize';

export type AlgorithmType = 'kmp' | 'boyer-moore' |
'regex' | 'fuzzy' | 'aho-corasick' | 'rabin-karp';

export interface PipelineMatch {
  keyword: string;
  index: number;
  algorithm: AlgorithmType;
}

```

```

        executionTime: number;
        comparisons?: number;
        matchText?: string;
    }

export interface AlgoStat {
    time: number;
    matches: number;
    comparisons: number;
}

export interface PipelineStats {
    totalMatches: number;
    algorithmStats: {
        kmp: AlgoStat;
        bm: AlgoStat;
        regex: AlgoStat;
        fuzzy: AlgoStat;
        ahoCorasick: AlgoStat;
        rabinKarp: AlgoStat;
    };
    matchesByAlgorithm: Record<string, number>;
    keywordBreakdown: Record<string, number>;
}

function isWholeWordMatch(text: string, index: number,
keywordLen: number): boolean {
    const before = index <= 0 ? true :
    !/[a-zA-Z0-9]/.test(text[index - 1]);
    const after = index + keywordLen >= text.length ?
    true : !/[a-zA-Z0-9]/.test(text[index + keywordLen]);
    return before && after;
}

function emptyStats(): PipelineStats {
    return {
        totalMatches: 0,
        algorithmStats: {
            kmp: { time: 0, matches: 0, comparisons: 0

```

```

    },
    bm: { time: 0, matches: 0, comparisons: 0 },
    regex: { time: 0, matches: 0, comparisons: 0 },
    fuzzy: { time: 0, matches: 0, comparisons: 0 },
    ahoCorasick: { time: 0, matches: 0, comparisons: 0 },
    rabinKarp: { time: 0, matches: 0, comparisons: 0 },
    matchesByAlgorithm: { kmp: 0, 'boyer-moore': 0, regex: 0, fuzzy: 0, 'aho-corasick': 0, 'rabin-karp': 0 },
    keywordBreakdown: {},
  };
}

export function runPipeline(
  text: string,
  keywords: string[],
  useKMP: boolean = true,
  useBM: boolean = true,
  useRegex: boolean = true,
  useFuzzy: boolean = true,
  useAC: boolean = false,
  useRK: boolean = false
): { matches: PipelineMatch[]; stats: PipelineStats } {
  const matches: PipelineMatch[] = [];
  const matchedSet = new Set<number>();
  const normalizedText = normalizeText(text);
  const stats = emptyStats();

  function addMatch(keyword: string, index: number, algo: AlgorithmType, time: number, comparisons?: number, matchText?: string) {
    if (matchedSet.has(index)) return;
    matchedSet.add(index);
  }

```

```

        matches.push({ keyword, index, algorithm: algo,
executionTime: time, comparisons, matchText });
        stats.keywordBreakdown[keyword] =
(stats.keywordBreakdown[keyword] || 0) + 1;
    }

    function filterExactMatches(
        algoResults: Map<string, MatchResult>,
        algo: AlgorithmType
    ): { cmp: number; cnt: number } {
        let cmp = 0, cnt = 0;
        for (const [keyword, r] of algoResults) {
            cmp += r.comparisons;
            for (const idx of r.indices) {
                if (keyword.length <= 4 &&
!isWholeWordMatch(normalizedText, idx, keyword.length))
continue;
                addMatch(keyword, idx, algo,
r.executionTime, r.comparisons);
                cnt++;
            }
        }
        return { cmp, cnt };
    }

    if (useKMP) {
        const t0 = performance.now();
        const results =
kmpSearchMultiple(normalizedText, keywords);
        const elapsed = performance.now() - t0;
        const { cmp, cnt } =
filterExactMatches(results, 'kmp');
        stats.algorithmStats.kmp = { time: elapsed,
matches: cnt, comparisons: cmp };
        stats.matchesByAlgorithm.kmp = cnt;
    }

    if (useBM) {
        const t0 = performance.now();

```

```

        const results =
bmSearchMultiple(normalizedText, keywords);
        const elapsed = performance.now() - t0;
        const { cmp, cnt } =
filterExactMatches(results, 'boyer-moore');
        stats.algorithmStats.bm = { time: elapsed,
matches: cnt, comparisons: cmp };
        stats.matchesByAlgorithm['boyer-moore'] = cnt;
    }

    if (useRegex) {
        const t0 = performance.now();
        const results = regexSearchAll(normalizedText);
        const elapsed = performance.now() - t0;
        for (const r of results) {
            addMatch(r.match, r.index, 'regex',
elapsed, 0, r.match);
        }
        stats.algorithmStats.regex = { time: elapsed,
matches: results.length, comparisons: 0 };
        stats.matchesByAlgorithm.regex =
results.length;
    }

    if (useAC) {
        const t0 = performance.now();
        const results =
ahoCorasickSearch(normalizedText, keywords);
        const elapsed = performance.now() - t0;
        const { cmp, cnt } =
filterExactMatches(results, 'aho-corasick');
        stats.algorithmStats.ahoCorasick = { time:
elapsed, matches: cnt, comparisons: cmp };
        stats.matchesByAlgorithm['aho-corasick'] = cnt;
    }

    if (useRK) {
        const t0 = performance.now();
        const results =

```

```

rabinKarpSearchMultiple(normalizedText, keywords);
    const elapsed = performance.now() - t0;
    const { cmp, cnt } =
filterExactMatches(results, 'rabin-karp');
    stats.algorithmStats.rabinKarp = { time:
elapsed, matches: cnt, comparisons: cmp };
    stats.matchesByAlgorithm['rabin-karp'] = cnt;
}

if (useFuzzy) {
    const exactMatchedKeywords = new Set<string>();
    for (const m of matches) {
        if (m.algorithm === 'kmp' || m.algorithm
=== 'boyer-moore' || m.algorithm === 'aho-corasick' ||
m.algorithm === 'rabin-karp') {
            exactMatchedKeywords.add(m.keyword);
        }
    }
    const fuzzyKeywords = keywords.filter(k =>
!exactMatchedKeywords.has(k));
    const t0 = performance.now();

    const fuzzyResults =
fuzzySearchAll(normalizedText, fuzzyKeywords);
    const elapsed = performance.now() - t0;
    let fuzzyCmp = fuzzyResults.length *
fuzzyKeywords.length;
    let fuzzyCnt = 0;

    for (const r of fuzzyResults) {
        const alreadyMatched =
matchedSet.has(r.startIndex);
        if (alreadyMatched) continue;
        addMatch(r.keyword, r.startIndex, 'fuzzy',
elapsed, 1, r.word);
        fuzzyCnt++;
    }

    stats.algorithmStats.fuzzy = { time: elapsed,

```

```

matches: fuzzyCnt, comparisons: fuzzyCmp };
    stats.matchesByAlgorithm.fuzzy = fuzzyCnt;
  }

  stats.totalMatches = matches.length;
  return { matches, stats };
}

export function deduplicateMatches(matches:
PipelineMatch[]): PipelineMatch[] {
  const seen = new Set<number>();
  return matches.filter(m => {
    if (seen.has(m.index)) return false;
    seen.add(m.index);
    return true;
  });
}

import { kmpSearchMultiple, MatchResult } from './kmp';
import { bmSearchMultiple } from './boyer-moore';
import { regexSearchAll } from './regex';
import { fuzzySearchAll } from './levenshtein';
import { ahoCorasickSearch } from './aho-corasick';
import { rabinKarpSearchMultiple } from './rabin-karp';
import { normalizeText } from './normalize';

export type AlgorithmType = 'kmp' | 'boyer-moore' |
'regex' | 'fuzzy' | 'aho-corasick' | 'rabin-karp';

export interface PipelineMatch {
  keyword: string;
  index: number;
  algorithm: AlgorithmType;
  executionTime: number;
  comparisons?: number;
  matchText?: string;
}

export interface AlgoStat {

```

```

        time: number;
        matches: number;
        comparisons: number;
    }

export interface PipelineStats {
    totalMatches: number;
    algorithmStats: {
        kmp: AlgoStat;
        bm: AlgoStat;
        regex: AlgoStat;
        fuzzy: AlgoStat;
        ahoCorasick: AlgoStat;
        rabinKarp: AlgoStat;
    };
    matchesByAlgorithm: Record<string, number>;
    keywordBreakdown: Record<string, number>;
}

function isWholeWordMatch(text: string, index: number,
keywordLen: number): boolean {
    const before = index <= 0 ? true :
    !/[a-zA-Z0-9]/.test(text[index - 1]);
    const after = index + keywordLen >= text.length ?
    true : !/[a-zA-Z0-9]/.test(text[index + keywordLen]);
    return before && after;
}

function emptyStats(): PipelineStats {
    return {
        totalMatches: 0,
        algorithmStats: {
            kmp: { time: 0, matches: 0, comparisons: 0
        },
            bm: { time: 0, matches: 0, comparisons: 0
        },
            regex: { time: 0, matches: 0, comparisons:
0 },
            fuzzy: { time: 0, matches: 0, comparisons:

```

```

0 },
    ahoCorasick: { time: 0, matches: 0,
comparisons: 0 },
    rabinKarp: { time: 0, matches: 0,
comparisons: 0 },
    },
    matchesByAlgorithm: { kmp: 0, 'boyer-moore': 0,
regex: 0, fuzzy: 0, 'aho-corasick': 0, 'rabin-karp': 0
},
    keywordBreakdown: {},
};
}

export function runPipeline(
    text: string,
    keywords: string[],
    useKMP: boolean = true,
    useBM: boolean = true,
    useRegex: boolean = true,
    useFuzzy: boolean = true,
    useAC: boolean = false,
    useRK: boolean = false
): { matches: PipelineMatch[]; stats: PipelineStats } {
    const matches: PipelineMatch[] = [];
    const matchedSet = new Set<number>();
    const normalizedText = normalizeText(text);
    const stats = emptyStats();

    function addMatch(keyword: string, index: number,
algo: AlgorithmType, time: number, comparisons?:
number, matchText?: string) {
        if (matchedSet.has(index)) return;
        matchedSet.add(index);
        matches.push({ keyword, index, algorithm: algo,
executionTime: time, comparisons, matchText });
        stats.keywordBreakdown[keyword] =
(stats.keywordBreakdown[keyword] || 0) + 1;
    }
}

```

```

function filterExactMatches(
  algoResults: Map<string, MatchResult>,
  algo: AlgorithmType
): { cmp: number; cnt: number } {
  let cmp = 0, cnt = 0;
  for (const [keyword, r] of algoResults) {
    cmp += r.comparisons;
    for (const idx of r.indices) {
      if (keyword.length <= 4 &&
!isWholeWordMatch(normalizedText, idx, keyword.length))
continue;
      addMatch(keyword, idx, algo,
r.executionTime, r.comparisons);
      cnt++;
    }
  }
  return { cmp, cnt };
}

if (useKMP) {
  const t0 = performance.now();
  const results =
kmpSearchMultiple(normalizedText, keywords);
  const elapsed = performance.now() - t0;
  const { cmp, cnt } =
filterExactMatches(results, 'kmp');
  stats.algorithmStats.kmp = { time: elapsed,
matches: cnt, comparisons: cmp };
  stats.matchesByAlgorithm.kmp = cnt;
}

if (useBM) {
  const t0 = performance.now();
  const results =
bmSearchMultiple(normalizedText, keywords);
  const elapsed = performance.now() - t0;
  const { cmp, cnt } =
filterExactMatches(results, 'boyer-moore');
  stats.algorithmStats.bm = { time: elapsed,

```

```

matches: cnt, comparisons: cmp };
    stats.matchesByAlgorithm['boyer-moore'] = cnt;
}

if (useRegex) {
    const t0 = performance.now();
    const results = regexSearchAll(normalizedText);
    const elapsed = performance.now() - t0;
    for (const r of results) {
        addMatch(r.match, r.index, 'regex',
elapsed, 0, r.match);
    }
    stats.algorithmStats.regex = { time: elapsed,
matches: results.length, comparisons: 0 };
    stats.matchesByAlgorithm.regex =
results.length;
}

if (useAC) {
    const t0 = performance.now();
    const results =
ahoCorasickSearch(normalizedText, keywords);
    const elapsed = performance.now() - t0;
    const { cmp, cnt } =
filterExactMatches(results, 'aho-corasick');
    stats.algorithmStats.ahoCorasick = { time:
elapsed, matches: cnt, comparisons: cmp };
    stats.matchesByAlgorithm['aho-corasick'] = cnt;
}

if (useRK) {
    const t0 = performance.now();
    const results =
rabinKarpSearchMultiple(normalizedText, keywords);
    const elapsed = performance.now() - t0;
    const { cmp, cnt } =
filterExactMatches(results, 'rabin-karp');
    stats.algorithmStats.rabinKarp = { time:
elapsed, matches: cnt, comparisons: cmp };

```

```

        stats.matchesByAlgorithm['rabin-karp'] = cnt;
    }

    if (useFuzzy) {
        const exactMatchedKeywords = new Set<string>();
        for (const m of matches) {
            if (m.algorithm === 'kmp' || m.algorithm
            === 'boyer-moore' || m.algorithm === 'aho-corasick' ||
            m.algorithm === 'rabin-karp') {
                exactMatchedKeywords.add(m.keyword);
            }
        }
        const fuzzyKeywords = keywords.filter(k =>
        !exactMatchedKeywords.has(k));
        const t0 = performance.now();

        const fuzzyResults =
        fuzzySearchAll(normalizedText, fuzzyKeywords);
        const elapsed = performance.now() - t0;
        let fuzzyCmp = fuzzyResults.length *
        fuzzyKeywords.length;
        let fuzzyCnt = 0;

        for (const r of fuzzyResults) {
            const alreadyMatched =
            matchedSet.has(r.startIndex);
            if (alreadyMatched) continue;
            addMatch(r.keyword, r.startIndex, 'fuzzy',
            elapsed, 1, r.word);
            fuzzyCnt++;
        }

        stats.algorithmStats.fuzzy = { time: elapsed,
        matches: fuzzyCnt, comparisons: fuzzyCmp };
        stats.matchesByAlgorithm.fuzzy = fuzzyCnt;
    }

    stats.totalMatches = matches.length;
    return { matches, stats };

```

```

}

export function deduplicateMatches(matches:
PipelineMatch[]): PipelineMatch[] {
  const seen = new Set<number>();
  return matches.filter(m => {
    if (seen.has(m.index)) return false;
    seen.add(m.index);
    return true;
  });
}

```

g. aho-corasick.ts

```

import { MatchResult } from './kmp';

interface AhoNode {
  children: Map<string, AhoNode>;
  fail: AhoNode | null;
  output: string[];
  depth: number;
}

function buildTrie(patterns: string[]): AhoNode {
  const root: AhoNode = {
    children: new Map(),
    fail: null,
    output: [],
    depth: 0,
  };

  for (const pattern of patterns) {
    let node = root;
    for (const ch of pattern) {
      const upper = ch.toUpperCase();
      if (!node.children.has(upper)) {
        node.children.set(upper, {
          children: new Map(),

```

```

        fail: null,
        output: [],
        depth: node.depth + 1,
    });
    }
    node = node.children.get(upper)!;
}
node.output.push(pattern);
}

return root;
}

function buildFailureLinks(root: AhoNode): void {
    const queue: AhoNode[] = [];

    for (const [, child] of root.children) {
        child.fail = root;
        queue.push(child);
    }

    while (queue.length > 0) {
        const node = queue.shift()!;

        for (const [ch, child] of node.children) {
            let fail = node.fail;
            while (fail !== null &&
!fail.children.has(ch)) {
                fail = fail.fail;
            }
            child.fail = fail ? fail.children.get(ch)!
: root;

            child.output.push(...child.fail.output);
            queue.push(child);
        }
    }
}

export function ahoCorasickSearch(text: string,

```

```

patterns: string[]): Map<string, MatchResult> {
    const startTime = performance.now();
    const results = new Map<string, MatchResult>();
    const upperText = text.toUpperCase();

    for (const p of patterns) {
        results.set(p, { indices: [], executionTime: 0,
comparisons: 0 });
    }

    const root = buildTrie(patterns);
    buildFailureLinks(root);

    let node = root;
    let comparisons = 0;

    for (let i = 0; i < upperText.length; i++) {
        const ch = upperText[i];

        while (node !== root && !node.children.has(ch))
        {
            node = node.fail!;
            comparisons++;
        }
        comparisons++;

        if (node.children.has(ch)) {
            node = node.children.get(ch)!;
        } else {
            continue;
        }

        if (node.output.length > 0) {
            for (const matched of node.output) {
                const result = results.get(matched);
                if (result) {
                    result.indices.push(i -
matched.length + 1);
                }
            }
        }
    }
}

```

```

    }
  }
}

const elapsed = performance.now() - startTime;
for (const [, result] of results) {
  result.executionTime = elapsed;
  result.comparisons = comparisons;
}

return results;
}

export interface AhoCorasickMatch {
  keyword: string;
  index: number;
  algorithm: 'aho-corasick';
  executionTime: number;
  comparisons: number;
}

export function ahoCorasickSearchFlat(text: string,
patterns: string[]): AhoCorasickMatch[] {
  const startTime = performance.now();
  const upperText = text.toUpperCase();

  const root = buildTrie(patterns);
  buildFailureLinks(root);

  const matches: AhoCorasickMatch[] = [];
  let node = root;
  let comparisons = 0;

  for (let i = 0; i < upperText.length; i++) {
    const ch = upperText[i];

    while (node !== root && !node.children.has(ch))
    {
      node = node.fail!;
    }
  }
}

```

```

        comparisons++;
    }
    comparisons++;

    if (node.children.has(ch)) {
        node = node.children.get(ch)!;
    } else {
        continue;
    }

    if (node.output.length > 0) {
        for (const matched of node.output) {
            matches.push({
                keyword: matched,
                index: i - matched.length + 1,
                algorithm: 'aho-corasick',
                executionTime: performance.now() -
startTime,
                comparisons,
            });
        }
    }
}

return matches;
}

```

h. rabin-karp.ts

```

import { MatchResult } from './kmp';

const BASE = 256;
const MOD = 1000000007;

export function rabinKarpSearch(text: string, pattern:
string): MatchResult {
    const indices: number[] = [];

```

```

let comparisons = 0;

const n = text.length;
const m = pattern.length;

if (m === 0 || m > n) {
    return { indices, executionTime: 0,
comparisons: 0 };
}

const startTime = performance.now();
const textUpper = text.toUpperCase();
const patternUpper = pattern.toUpperCase();

let patHash = 0;
let txtHash = 0;
let h = 1;

for (let i = 0; i < m - 1; i++) {
    h = (h * BASE) % MOD;
}

for (let i = 0; i < m; i++) {
    patHash = (BASE * patHash +
patternUpper.charCodeAt(i)) % MOD;
    txtHash = (BASE * txtHash +
textUpper.charCodeAt(i)) % MOD;
}

for (let i = 0; i <= n - m; i++) {
    comparisons++;
    if (patHash === txtHash) {
        let j = 0;
        while (j < m && textUpper[i + j] ===
patternUpper[j]) {
            comparisons++;
            j++;
        }
        if (j === m) {

```

```

        indices.push(i);
    }
}

if (i < n - m) {
    txtHash = (
        BASE * (txtHash -
textUpper.charCodeAt(i) * h) +
        textUpper.charCodeAt(i + m)
    ) % MOD;
    if (txtHash < 0) txtHash += MOD;
}
}

return {
    indices,
    executionTime: performance.now() - startTime,
    comparisons,
};
}

export function rabinKarpSearchMultiple(text: string,
patterns: string[]): Map<string, MatchResult> {
    const results = new Map<string, MatchResult>();

    for (const pattern of patterns) {
        const result = rabinKarpSearch(text, pattern);
        if (result.indices.length > 0) {
            results.set(pattern, result);
        }
    }

    return results;
}

export interface RabinKarpMatch {
    keyword: string;
    index: number;
    algorithm: 'rabin-karp';
}

```

```

        executionTime: number;
        comparisons: number;
    }

    export function rabinKarpSearchFlat(text: string,
    patterns: string[]): RabinKarpMatch[] {
        const matches: RabinKarpMatch[] = [];

        for (const pattern of patterns) {
            const result = rabinKarpSearch(text, pattern);
            for (const idx of result.indices) {
                matches.push({
                    keyword: pattern,
                    index: idx,
                    algorithm: 'rabin-karp',
                    executionTime: result.executionTime,
                    comparisons: result.comparisons,
                });
            }
        }

        return matches;
    }

```

ii. Content

a. blur.ts

```

declare const chrome: any;

const BLUR_CLASS = 'judol-blurred';
let blurEnabled = false;
export function isBlurEnabled(): boolean {
    return blurEnabled;
}

```

```

}

export function setBlurEnabled(enabled: boolean): void
{
    blurEnabled = enabled;
    chrome.storage.local.set({ judo1BlurEnabled:
enabled });
}

export function blurDetectedElements(): void {
    const highlights =
document.querySelectorAll('.judo1-highlight');
    for (const el of highlights) {
        const parent = el.parentNode as HTMLElement |
null;
        if (!parent ||
parent.classList.contains(BLUR_CLASS)) continue;

        const block = findBlockContainer(el as
HTMLElement);
        if (!block ||
block.classList.contains(BLUR_CLASS)) continue;

        if (block.tagName === 'BODY' || block.tagName
=== 'HTML' || block.tagName === 'MAIN' ||
        block.closest('form') ||
block.closest('header') || block.closest('nav') ||
        block.closest('[role="search"]') ||
block.closest('input') || block.closest('textarea')) {
            continue;
        }

        block.classList.add(BLUR_CLASS);
        block.style.setProperty('filter', 'blur(6px)',
'important');
        block.style.cursor = 'pointer';
        block.style.transition = 'filter 0.3s ease';
    }
}

```

```

        const unblurOnClick = (e: MouseEvent) => {
            e.preventDefault();
            e.stopPropagation();
            unblurBlock(block);
            block.removeEventListener('click',
unblurOnClick);

            };
            block.addEventListener('click', unblurOnClick);
        }
    }

function unblurBlock(block: HTMLElement) {
    block.classList.remove(BLUR_CLASS);
    block.style.removeProperty('filter');
    block.style.cursor = '';
    block.style.transition = '';
}

export function unblurAll(): void {
    const blurred =
document.querySelectorAll(`.${BLUR_CLASS}`);
    for (const el of blurred) {
        unblurBlock(el as HTMLElement);
    }
}

function findBlockContainer(el: HTMLElement):
HTMLElement | null {
    let current: HTMLElement | null = el;
    for (let i = 0; i < 6; i++) {
        if (!current) break;

        if
(current.classList.contains('judol-highlight')) {
            current = current.parentElement;
            continue;
        }
    }
}

```

```

        const tag = current.tagName.toLowerCase();
        if (
            tag === 'p' || tag === 'div' || tag ===
'section' ||
            tag === 'article' || tag === 'li' || tag
=== 'td' ||
            tag === 'h1' || tag === 'h2' || tag ===
'h3' || tag === 'h4' || tag === 'h5' || tag === 'h6'
        ) {
            const rect =
current.getBoundingClientRect();
            if (rect.width > 10 && rect.height >= 10) {
                return current;
            }
        }
        current = current.parentElement;
    }
    return el.parentElement;
}

chrome.storage.local.get('judolBlurEnabled', (result:
any) => {
    blurEnabled = result.judolBlurEnabled === true;
});

```

b. content.ts

```

declare const chrome: any;

import { scanPage, clearHighlights } from './context';
import { blurDetectedElements, unblurAll,
setBlurEnabled as updateBlurState } from './blur';
import { setOcrEnabled as updateOcrState, scanImages }
from './ocr';

let scanTimeout: ReturnType<typeof setTimeout> | null =
null;

```

```

function init() {
    chrome.storage.local.get(
        ['judolBlurEnabled', 'judolOcrEnabled',
'judolAutoScan'],
        () => {
            scheduleScan(500);
        }
    );
}

function scheduleScan(delay: number = 300) {
    if (scanTimeout) clearTimeout(scanTimeout);
    scanTimeout = setTimeout(() => {
        performScan();
    }, delay);
}

async function performScan() {
    const stats = await scanPage();

    chrome.storage.local.get(['judolBlurEnabled',
'judolOcrEnabled'], (result: any) => {
        if (result.judolBlurEnabled)
blurDetectedElements();
        if (result.judolOcrEnabled) scanImages();
    });

    chrome.runtime.sendMessage({ type:
'JUDOL_SCAN_COMPLETE', stats });
}

chrome.runtime.onMessage.addListener((msg: any,
_sender: any, sendResponse: any) => {
    switch (msg.type) {
        case 'JUDOL_SCAN':
            performScan().then(() => sendResponse({
success: true }));
            return true;
    }
}

```

```
case 'JUDOL_CLEAR':
    clearHighlights();
    unblurAll();
    sendResponse({ success: true });
    return true;

case 'JUDOL_TOGGLE_BLUR':
    updateBlurState(msg.enabled);
    if (msg.enabled) {
        blurDetectedElements();
    } else {
        unblurAll();
    }
    sendResponse({ success: true });
    return true;

case 'JUDOL_TOGGLE_OCR':
    updateOcrState(msg.enabled);
    if (msg.enabled) {
        scanImages();
    }
    sendResponse({ success: true });
    return true;

case 'JUDOL_GET_STATS': {
    chrome.storage.local.get('judolStats',
(result: any) => {
        sendResponse({ stats: result.judolStats
|| null });
    });
    return true;
}

case 'JUDOL_PING':
    sendResponse({ pong: true });
    return true;
}
});
```

```
if (document.readyState === 'loading') {
    document.addEventListener('DOMContentLoaded',
init);
} else {
    init();
}
```

c. context.ts

```
import { PipelineMatch, PipelineStats, runPipeline,
deduplicateMatches } from '../algorithms/pipeline';
import { scanDOM } from './scanner';
import { highlightMatch, removeHighlights } from
'./highlighter';
import { attachTooltipEvents, detachTooltipEvents }
from './tooltip';
import { loadKeywordsFromFile } from './keywordLoader';

declare const chrome: any;

let isScanning = false;
let lastStats: PipelineStats | null = null;

function matchMapFromMatches(matches: PipelineMatch[]):
Map<number, PipelineMatch> {
    const matchMap = new Map<number, PipelineMatch>();
    const sorted = [...matches].sort((a, b) => a.index
- b.index);
    for (const m of sorted) {
        if (!matchMap.has(m.index)) {
            matchMap.set(m.index, m);
        }
    }
    return matchMap;
}

function containsManual(text: string, pattern: string):
```

```

boolean {
    if (pattern.length === 0) return false;
    for (let i = 0; i <= text.length - pattern.length;
i++) {
        let match = true;
        for (let j = 0; j < pattern.length; j++) {
            if (text[i + j] !== pattern[j]) { match =
false; break; }
        }
        if (match) return true;
    }
    return false;
}

export async function scanPage(): Promise<PipelineStats
| null> {
    if (isScanning) return null;
    isScanning = true;

    try {
        removeHighlights();
        detachTooltipEvents();

        const keywords = await loadKeywordsFromFile();
        const segments = scanDOM();
        const fullText = segments.map(s =>
s.text).join('\n');

        if (fullText.trim().length === 0) {
            isScanning = false;
            return null;
        }

        const algoPrefs = await new Promise<{
            useAC: boolean; useRK: boolean;
        }>((resolve) => {
            chrome.storage.local.get(
                ['judolUseAC', 'judolUseRK'],
                (r: any) => resolve({

```

```

        useAC: r.judolUseAC === true,
        useRK: r.judolUseRK === true,
    })
    );
});

const { matches, stats } = runPipeline(
    fullText,
    keywords,
    true,
    true,
    true,
    true,
    algoPrefs.useAC,
    algoPrefs.useRK
);

const uniqueMatches =
deduplicateMatches(matches);
const matchMap =
matchMapFromMatches(uniqueMatches);

let globalOffset = 0;
for (const seg of segments) {
    const segLen = seg.text.length;

    const segMatches: { match: PipelineMatch;
localOffset: number }[] = [];
    for (const [globalIdx, m] of matchMap) {
        if (globalIdx >= globalOffset &&
globalIdx < globalOffset + segLen) {
            segMatches.push({ match: m,
localOffset: globalIdx - globalOffset });
        }
    }

    segMatches.sort((a, b) => b.localOffset -
a.localOffset);

```

```

        let currentTextNode: Text = seg.node;
        for (const sm of segMatches) {
            const matchText = sm.match.matchText ||
sm.match.keyword;
            const endOffset = sm.localOffset +
matchText.length;

            if (endOffset <=
(currentTextNode.textContent || '').length) {
                highlightMatch(currentTextNode,
sm.localOffset, endOffset, {
                    keyword: sm.match.keyword,
                    algorithm: sm.match.algorithm,
                    executionTime:
sm.match.executionTime,
                    comparisons:
sm.match.comparisons,
                    matchText: matchText,
                });
            }

            const parent =
currentTextNode.parentNode;
            if (parent) parent.normalize();

            const walker =
document.createTreeWalker(
                seg.node.parentNode ||
document.body,
                NodeFilter.SHOW_TEXT
            );
            let found: Text | null = null;
            let n: Text | null;
            while ((n = walker.nextNode() as Text |
null) !== null) {
                const tc = n.textContent || '';
                if (containsManual(tc, matchText))
{
                    found = n;

```

```

                break;
            }
        }
        if (found) currentTextNode = found;
    }

    globalOffset += segLen + 1;
}

attachTooltipEvents();
lastStats = stats;

chrome.storage.local.set({ judolStats: stats,
judolLastScan: Date.now() });

return stats;
} catch (e) {
    console.error('Judol Detector scan error:', e);
    return null;
} finally {
    isScanning = false;
}
}

export function clearHighlights(): void {
    removeHighlights();
    detachTooltipEvents();
    lastStats = null;
    chrome.storage.local.set({ judolStats: null });
}

export function getLastStats(): PipelineStats | null {
    return lastStats;
}

```

d. highlighter.ts

```

export interface HighlightInfo {
    keyword: string;
    algorithm: string;
    executionTime: number;
    comparisons?: number;
    matchText?: string;
}

const HIGHLIGHT_CLASS = 'judol-highlight';
let highlightIdCounter = 0;

export function highlightMatch(
    textNode: Text,
    startOffset: number,
    endOffset: number,
    info: HighlightInfo
): HTMLElement | null {
    const parent = textNode.parentNode;
    if (!parent) return null;

    const fullText = textNode.textContent || '';

    const before = fullText.substring(0, startOffset);
    const match = fullText.substring(startOffset,
endOffset);
    const after = fullText.substring(endOffset);

    const span = document.createElement('span');
    span.className = HIGHLIGHT_CLASS;
    span.dataset.judolKeyword = info.keyword;
    span.dataset.judolAlgorithm = info.algorithm;
    span.dataset.judolTime =
String(info.executionTime);
    span.dataset.judolComparisons =
String(info.comparisons ?? 0);
    span.dataset.judolMatchText = info.matchText ||
match;
    span.dataset.judolId =
String(++highlightIdCounter);

```

```

    span.textContent = match;

    const fragment = document.createDocumentFragment();
    if (before)
fragment.appendChild(document.createTextNode(before));
    fragment.appendChild(span);
    if (after)
fragment.appendChild(document.createTextNode(after));

    parent.replaceChild(fragment, textNode);

    return span;
}

export function removeHighlights(): void {
    const highlights =
document.querySelectorAll(`.${HIGHLIGHT_CLASS}`);
    for (const el of highlights) {
        const parent = el.parentNode;
        if (!parent) continue;
        const text =
document.createTextNode(el.textContent || '');
        parent.replaceChild(text, el);
        parent.normalize();
    }
}

export function getHighlightCount(): number {
    return
document.querySelectorAll(`.${HIGHLIGHT_CLASS}`).length
;
}

```

e. keywordLoader.ts

```

let cached: string[] | null = null;

```

```

declare const chrome: any;

export async function loadKeywordsFromFile():
Promise<string[]> {
  if (cached !== null) return cached;

  try {
    const url =
chrome.runtime.getURL('keywords.txt');
    const resp = await fetch(url);
    const text = await resp.text();
    const keywords = text
      .split('\n')
      .map(k => k.trim().toLowerCase())
      .filter(k => k.length > 0);

    if (keywords.length > 0) {
      cached = keywords;
      return keywords;
    }
  } catch (e) {
    console.warn('Failed to load keywords.txt:',
e);
  }

  cached = [];
  return cached;
}

```

f. ocr.ts

```

import { loadKeywordsFromFile } from './keywordLoader';

declare const chrome: any;

let ocrEnabled = false;
let ocrKeywords: string[] | null = null;

```

```
export function isOcrEnabled(): boolean {
    return ocrEnabled;
}

export function setOcrEnabled(enabled: boolean): void {
    ocrEnabled = enabled;
    chrome.storage.local.set({ judolOcrEnabled: enabled
});
}

export async function scanImages(): Promise<void> {
    if (!ocrEnabled) return;

    let Tesseract: any;
    try {
        Tesseract = await import('tesseract.js');
    } catch {
        console.warn('Tesseract.js not available,
skipping OCR');
        return;
    }

    if (!ocrKeywords) {
        ocrKeywords = await loadKeywordsFromFile();
    }
    if (!ocrKeywords || ocrKeywords.length === 0)
return;

    const images = document.querySelectorAll('img');
    for (const img of images) {
        if (img.dataset.judolScanned === 'true')
continue;
        if (!img.src) continue;
        if (img.width < 50 || img.height < 50)
continue;
        if (img.complete === false) continue;

        img.dataset.judolScanned = 'true';
    }
}
```

```

    try {
        const canvas =
document.createElement('canvas');
        canvas.width = img.naturalWidth ||
img.width;
        canvas.height = img.naturalHeight ||
img.height;
        const ctx = canvas.getContext('2d');
        if (!ctx) continue;

        const clone = new Image();
        clone.crossOrigin = 'anonymous';

        await new Promise((resolve, reject) => {
            clone.onload = resolve;
            clone.onerror = reject;
            clone.src = img.src;
        });

        ctx.drawImage(clone, 0, 0);
        const dataUrl =
canvas.toDataURL('image/png');

        const result: any = await
Tesseract.recognize(dataUrl, 'eng', {
            logger: () => {},
        });

        const detectedText: string =
(result?.data?.text || '').toUpperCase();
        let detected = false;

        for (const keyword of ocrKeywords) {
            const upperKw = keyword.toUpperCase();
            if (containsManual(detectedText,
upperKw)) {
                detected = true;
                break;
            }
        }
    }

```

```

    }

    if (!detected) {
        const judolPattern =
/[A-Z]{3,}\d{2,3}|\d{2,3}[A-Z]{3,}/;
        if (judolPattern.test(detectedText)) {
            detected = true;
        }
    }

    if (detected) {
        img.style.filter = 'blur(8px)';
        img.dataset.judolBlocked = 'true';
    }
} catch (e) {
    console.warn('OCR failed for image:', e);
}
}

function containsManual(text: string, pattern: string):
boolean {
    if (pattern.length === 0) return false;
    for (let i = 0; i <= text.length - pattern.length;
i++) {
        let match = true;
        for (let j = 0; j < pattern.length; j++) {
            if (text[i + j] !== pattern[j]) { match =
false; break; }
        }
        if (match) return true;
    }
    return false;
}

chrome.storage.local.get('judolOcrEnabled', (result:
any) => {
    ocrEnabled = result.judolOcrEnabled === true;
});

```

g. scanner.ts

```
export interface TextSegment {
  text: string;
  node: Text;
  startOffset: number;
}

export function scanDOM(root: Node = document.body):
TextSegment[] {
  const segments: TextSegment[] = [];
  const walker = document.createTreeWalker(
    root,
    NodeFilter.SHOW_TEXT,
    {
      acceptNode(node) {
        const parent = (node as
Text).parentElement;
        if (!parent) return
NodeFilter.FILTER_REJECT;
        const tag =
parent.tagName.toLowerCase();
        if (
          tag === 'script' || tag === 'style'
|| tag === 'noscript' ||
          tag === 'iframe' || tag === 'svg'
|| tag === 'canvas' ||
          tag === 'textarea' || tag ===
'option'
        ) {
          return NodeFilter.FILTER_REJECT;
        }
        const text = (node as Text).textContent
|| '';
        if (text.trim().length === 0) return
NodeFilter.FILTER_REJECT;
```

```

        return NodeFilter.FILTER_ACCEPT;
    },
    }
);

let node: Text | null;
while ((node = walker.nextNode() as Text | null)
!== null) {
    const text = node.textContent || '';
    if (text.trim().length > 0) {
        segments.push({ text, node, startOffset: 0
});
    }
}

return segments;
}

```

h. tooltip.ts

```

const TOOLTIP_CLASS = 'judol-tooltip';

let tooltipEl: HTMLElement | null = null;
const handlerMap = new WeakMap<HTMLElement, { enter:
(e: Event) => void; leave: () => void }>();

function countKeywordOccurrences(keyword: string):
number {
    const highlights =
document.querySelectorAll('.judol-highlight');
    let count = 0;
    for (const el of highlights) {
        if ((el as HTMLElement).dataset.judolKeyword
=== keyword) count++;
    }
    return count;
}

```

```

function countAlgorithmOccurrences(algo: string):
number {
    const highlights =
document.querySelectorAll('.judol-highlight');
    let count = 0;
    for (const el of highlights) {
        if ((el as HTMLElement).dataset.judolAlgorithm
=== algo) count++;
    }
    return count;
}

function ensureTooltip(): HTMLElement {
    if (tooltipEl) return tooltipEl;

    tooltipEl = document.createElement('div');
    tooltipEl.className = TOOLTIP_CLASS;
    tooltipEl.style.cssText = `
        display: none;
        position: fixed;
        z-index: 2147483647;
        background: #1a1a2e;
        color: #eee;
        padding: 12px 16px;
        border-radius: 8px;
        font-family: 'Segoe UI', Arial, sans-serif;
        font-size: 13px;
        line-height: 1.5;
        box-shadow: 0 4px 20px rgba(0,0,0,0.5);
        border: 1px solid rgba(255,255,255,0.1);
        max-width: 320px;
        pointer-events: none;
    `;
    document.body.appendChild(tooltipEl);
    return tooltipEl;
}

function showTooltip(_e: MouseEvent, el: HTMLElement) {

```

```

    const keyword = el.dataset.judolKeyword || '';
    const algorithm = el.dataset.judolAlgorithm || '';
    const time = el.dataset.judolTime || '0';
    const comparisons = el.dataset.judolComparisons ||
'0';

    const matchText = el.dataset.judolMatchText ||
el.textContent || '';
    const totalOccurrences =
countKeywordOccurrences(keyword);
    const algoOccurrences =
countAlgorithmOccurrences(algorithm);

    const tooltip = ensureTooltip();
    tooltip.innerHTML = `
        <div
style="font-weight:700;color:#ff6b6b;margin-bottom:6px;
font-size:14px;">
            Judol Terdeteksi!
        </div>
        <table
style="width:100%;border-collapse:collapse;">
            <tr><td style="padding:2px 8px 2px
0;color:#aaa;">Keyword</td>
                <td style="padding:2px
0;color:#fff;font-weight:500;">${escapeHtml(keyword)}</
td></tr>
            <tr><td style="padding:2px 8px 2px
0;color:#aaa;">Match</td>
                <td style="padding:2px
0;color:#ffd93d;">${escapeHtml(matchText)}</td></tr>
            <tr><td style="padding:2px 8px 2px
0;color:#aaa;">Algorithm</td>
                <td style="padding:2px
0;color:#6bcbff;">${escapeHtml(algorithm)}</td></tr>
            <tr><td style="padding:2px 8px 2px
0;color:#aaa;">Occurrences</td>
                <td style="padding:2px
0;color:#6bcbff;">${totalOccurrences} total /
${algoOccurrences} by algorithm</td></tr>

```

```

        <tr><td style="padding:2px 8px 2px
0;color:#aaa;">Time</td>
        <td style="padding:2px
0;color:#6bcbff;">${escapeHtml(time)} ms</td></tr>
        <tr><td style="padding:2px 8px 2px
0;color:#aaa;">Comparisons</td>
        <td style="padding:2px
0;color:#6bcbff;">${escapeHtml(comparisons)}</td></tr>
    </table>
    `;

    const rect = el.getBoundingClientRect();
    let top = rect.bottom + 8;
    let left = rect.left;

    if (top + tooltip.offsetHeight >
window.innerHeight) {
        top = rect.top - tooltip.offsetHeight - 8;
    }
    if (left + tooltip.offsetWidth > window.innerWidth)
    {
        left = window.innerWidth - tooltip.offsetWidth
- 8;
    }
    if (left < 8) left = 8;

    tooltip.style.top = `${top}px`;
    tooltip.style.left = `${left}px`;
    tooltip.style.display = 'block';
}

function hideTooltip() {
    if (tooltipEl) {
        tooltipEl.style.display = 'none';
    }
}

function escapeHtml(str: string): string {
    return str

```

```

        .replace(/&/g, '&amp;');
        .replace(/</g, '&lt;');
        .replace(/>/g, '&gt;');
        .replace(/"/g, '&quot;');
        .replace(/'/g, '&#039;');
    }

export function attachTooltipEvents(): void {
    const highlights =
document.querySelectorAll('.judol-highlight');
    for (const el of highlights) {
        const enter = (e: Event) => showTooltip(e as
MouseEvent, el as HTMLElement);
        const leave = () => hideTooltip();
        el.addEventListener('mouseenter', enter);
        el.addEventListener('mouseleave', leave);
        handlerMap.set(el as HTMLElement, { enter,
leave });
    }
}

export function detachTooltipEvents(): void {
    const highlights =
document.querySelectorAll('.judol-highlight');
    for (const el of highlights) {
        const handlers = handlerMap.get(el as
HTMLElement);
        if (handlers) {
            el.removeEventListener('mouseenter',
handlers.enter);
            el.removeEventListener('mouseleave',
handlers.leave);
            handlerMap.delete(el as HTMLElement);
        }
    }
}

```

iii. Popup

a. App.tsx

```
import { useState, useEffect, useCallback } from
"react";
import type { PipelineStats, AlgorithmType } from
"../algorithms/pipeline";

declare const chrome: any;

const ALGO_LABELS: Record<string, string> = {
  kmp: "KMP",
  "boyer-moore": "Boyer-Moore",
  regex: "RegEx",
  fuzzy: "Fuzzy",
  "aho-corasick": "Aho-Corasick",
  "rabin-karp": "Rabin-Karp",
};

const ALGO_COLORS = [
  "#ff6b6b",
  "#4ecdc4",
  "#45b7d1",
  "#f9ca24",
  "#a29bfe",
  "#fd79a8",
];

async function getActiveTabId(): Promise<number |
undefined> {
  return new Promise((resolve) => {
    chrome.tabs.query({ active: true, currentWindow:
true }, (tabs: any) => {
      resolve(tabs[0]?.id);
    });
  });
}
```

```
function BarChart({
  data,
  labels,
  colors,
  title,
}): {
  data: number[];
  labels: string[];
  colors: string[];
  title: string;
}) {
  const max = Math.max(...data, 1);
  return (
    <div style={{ marginTop: 12 }}>
      <div
        style={{
          fontSize: 13,
          fontWeight: 600,
          color: "#ccc",
          marginBottom: 8,
        }}
      >
        {title}
      </div>
      {labels.map((label, i) => (
        <div
          key={label}
          style={{
            marginBottom: 6,
            display: "flex",
            alignItems: "center",
            gap: 8,
          }}
        >
          <div
            style={{
              width: 90,
              fontSize: 11,
              color: "#aaa",
```

```

        textAlign: "right",
        flexShrink: 0,
    }}
    >
    {label}
</div>
<div
  style={{
    flex: 1,
    height: 18,
    background: "rgba(255,255,255,0.05)",
    borderRadius: 4,
    overflow: "hidden",
    position: "relative",
  }}
  >
  <div
    style={{
      width: `${(data[i] / max) * 100}%`,
      height: "100%",
      background: colors[i % colors.length],
      borderRadius: 4,
      transition: "width 0.4s ease",
      minWidth: data[i] > 0 ? 4 : 0,
    }}
    />
  </div>
  <div
    style={{
      width: 32,
      fontSize: 11,
      color: "#fff",
      textAlign: "right",
    }}
    >
    {data[i]}
  </div>
</div>
)))}

```

```

    </div>
  );
}

const ALGO_STAT_KEY: Record<string, keyof
PipelineStats["algorithmStats"]> = {
  kmp: "kmp",
  "boyer-moore": "bm",
  regex: "regex",
  fuzzy: "fuzzy",
  "aho-corasick": "ahoCorasick",
  "rabin-karp": "rabinKarp",
};

function getAlgoName(algo: string): string {
  return ALGO_LABELS[algo] || algo;
}

function getAlgoMatches(stats: PipelineStats, algo:
AlgorithmType): number {
  return stats.matchesByAlgorithm[algo] || 0;
}

function getAlgoTime(stats: PipelineStats, algo:
AlgorithmType): number {
  const key = ALGO_STAT_KEY[algo];
  return key ? stats.algorithmStats[key].time : 0;
}

function getAlgoComparisons(stats: PipelineStats, algo:
AlgorithmType): number {
  const key = ALGO_STAT_KEY[algo];
  return key ? stats.algorithmStats[key].comparisons :
0;
}

export default function PopupApp() {
  const [stats, setStats] = useState<PipelineStats |
null>(null);

```

```

const [loading, setLoading] = useState(true);
const [blurEnabled, setBlurEnabled] =
useState(false);
const [ocrEnabled, setOcrEnabled] = useState(false);
const [acEnabled, setAcEnabled] = useState(false);
const [rkEnabled, setRkEnabled] = useState(false);
const [scanning, setScanning] = useState(false);
const [error, setError] = useState<string |
null>(null);

const loadStats = useCallback(async () => {
  try {
    const tabId = await getActiveTabId();
    if (!tabId) {
      setError("No active tab");
      setLoading(false);
      return;
    }
    const resp = await chrome.tabs.sendMessage(tabId,
{
  type: "JUDOL_GET_STATS",
});
    if (resp?.stats) {
      setStats(resp.stats as PipelineStats);
    } else {
      chrome.storage.local.get("judolStats", (result:
any) => {
        if (result.judolStats)
setStats(result.judolStats as PipelineStats);
      });
    }
  } catch {
    setError("Cannot connect to page");
  }
  setLoading(false);
}, []);

useEffect(() => {
  loadStats();

```

```

    chrome.storage.local.get(
        ["judolBlurEnabled", "judolOcrEnabled",
        "judolUseAC", "judolUseRK"],
        (result: any) => {
            setBlurEnabled(result.judolBlurEnabled ===
true);
            setOcrEnabled(result.judolOcrEnabled === true);
            setAcEnabled(result.judolUseAC === true);
            setRkEnabled(result.judolUseRK === true);
        },
    );

    const interval = setInterval(loadStats, 2000);
    return () => clearInterval(interval);
}, [loadStats]);

const handleScan = async () => {
    setScanning(true);
    setError(null);
    try {
        const tabId = await getActiveTabId();
        if (tabId) {
            await chrome.tabs.sendMessage(tabId, { type:
"JUDOL_SCAN" });
            setTimeout(async () => {
                await loadStats();
                setScanning(false);
            }, 500);
        }
    }
}

```

```

.....
btn: {
    flex: 1,
    padding: "8px 16px",
    border: "none",
    borderRadius: 6,
    cursor: "pointer",
    fontSize: 13,
    fontWeight: 600,
    transition: "all 0.2s",
},

```

```

    btnPrimary: {
      background: "linear-gradient(135deg, #6bcbff,
#0984e3) ",
      color: "#fff",
    },
    btnSecondary: {
      background: "rgba(255,255,255,0.1)",
      color: "#ccc",
    },
    toggles: {
      display: "flex",
      gap: 16,
      marginTop: 12,
      fontSize: 12,
    },
    toggleLabel: {
      display: "flex",
      alignItems: "center",
      gap: 6,
      cursor: "pointer",
    },
    toggleInput: {
      accentColor: "#6bcbff",
    },
    toggleText: {
      color: "#aaa",
    },
  },
};

```

iv. Styles

a. Content.css

```

.judol-highlight {
  background: rgba(255, 107, 107, 0.3) !important;
  border-bottom: 2px solid #ff6b6b !important;
  border-radius: 2px !important;
}

```

```
padding: 0 2px !important;
cursor: pointer !important;
transition: background 0.2s ease !important;
}

.judol-highlight:hover {
    background: rgba(255, 107, 107, 0.5) !important;
}

.judol-tooltip {
    animation: judolFadeIn 0.15s ease-out;
}

@keyframes judolFadeIn {
    from {
        opacity: 0;
        transform: translateY(4px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

.judol-blurred {
    filter: blur(8px) !important;
    transition: filter 0.3s ease !important;
    position: relative !important;
}

.judol-blur-label {
    position: absolute !important;
    top: 50% !important;
    left: 50% !important;
    transform: translate(-50%, -50%) !important;
    z-index: 2147483646 !important;
    background: rgba(0, 0, 0, 0.85) !important;
    color: #fff !important;
    padding: 8px 16px !important;
```

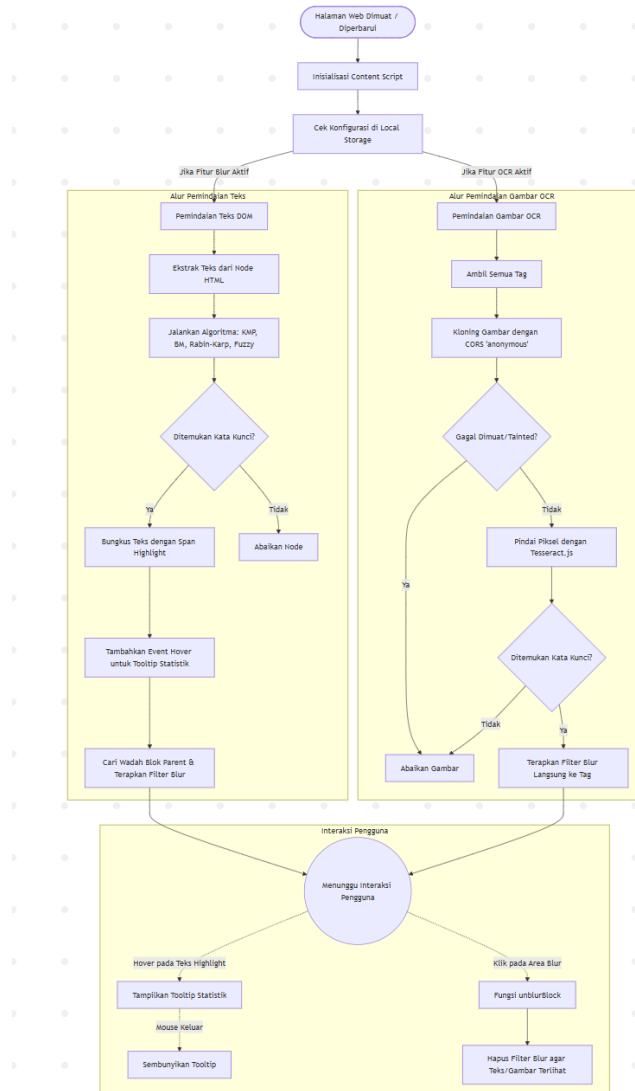
```
border-radius: 6px !important;
font-size: 13px !important;
font-family: Arial, sans-serif !important;
pointer-events: none !important;
white-space: nowrap !important;
}
```

b. index.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Segoe UI', system-ui, Avenir,
Helvetica, Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}
```

b. Alur Sistem secara Ringkas



Secara garis besar, ekstensi Judol Detector beroperasi secara otomatis begitu sebuah halaman web selesai dimuat di peramban pengguna. Tahap pertama yang dilakukan oleh sistem adalah tahap Inisialisasi, di mana content script membaca konfigurasi dari penyimpanan lokal browser (`chrome.storage.local`) untuk mengetahui preferensi pengguna apakah fitur penyensoran teks (Blur) dan pemindaian gambar (OCR) sedang dalam keadaan aktif atau tidak.

Jika fitur sensor teks aktif, sistem akan masuk ke tahap Pemindaian Teks (Text Processing). Ekstensi akan mengekstrak seluruh teks yang ada di dalam struktur HTML (DOM) halaman tersebut. Teks yang ditemukan kemudian diproses

menggunakan empat algoritma pencocokan string secara simultan, yaitu algoritma KMP (Knuth-Morris-Pratt), Boyer-Moore, Rabin-Karp, dan Fuzzy Matching. Jika algoritma mendeteksi adanya kata kunci yang terkait dengan perjudian online, sistem akan membungkus kata tersebut dengan sebuah penanda khusus (highlight) dan menyiapkan sebuah tooltip yang berisi data statistik performa dari algoritma yang digunakan. Setelah itu, sistem secara cerdas akan menelusuri hierarki elemen web tersebut ke atas untuk menemukan kotak pembungkus utamanya (misalnya keseluruhan satu paragraf, satu kontainer hasil pencarian, atau satu judul) dan menerapkan efek blur pada seluruh kotak pembungkus tersebut agar konten perjudian tidak langsung terlihat.

Bersamaan dengan itu, jika pengguna mengaktifkan fitur OCR, sistem akan menjalankan tahap Pemindaian Gambar (Image Processing). Ekstensi mencari seluruh elemen gambar () yang berukuran cukup besar di dalam halaman web. Untuk mencegah pemblokiran keamanan lintas domain (CORS / Tainted Canvas) yang sering terjadi pada gambar eksternal, sistem akan melakukan cloning (penggandaan) terhadap gambar tersebut secara anonim. Setelah gambar berhasil dikloning, piksel-piksel gambar tersebut dipindai menggunakan library Tesseract.js untuk mengekstrak teks yang ada di dalamnya. Jika teks yang terbaca dari dalam gambar tersebut mengandung kata kunci perjudian, sistem akan langsung menimpa gambar asli dengan filter blur yang tebal.

Pada akhirnya, ekstensi memberikan kendali kepada pengguna melalui tahap Interaksi Pengguna (User Interaction). Kotak teks maupun gambar yang telah diblur tidak terkunci secara permanen. Pengguna diberikan opsi untuk mengeklik secara langsung area yang diblur tersebut untuk menghapus sensornya, sehingga konten aslinya kembali terlihat. Selain itu, ketika pengguna mengarahkan kursor (hover) ke kata kunci judi yang telah diberi highlight, sistem akan memunculkan sebuah tooltip informasi yang merangkum hasil kinerja algoritma pencarian.

- c. Source Code: https://github.com/SpicyCoinCracker/Tubes3_NantiBisaDiganti

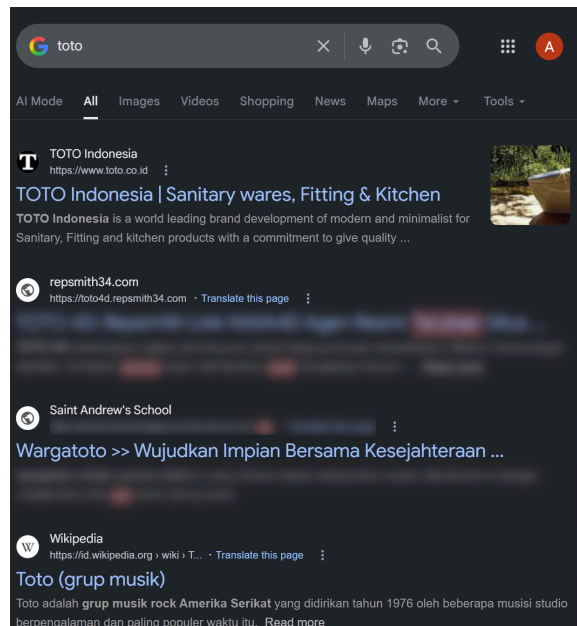
4.2. Pengujian

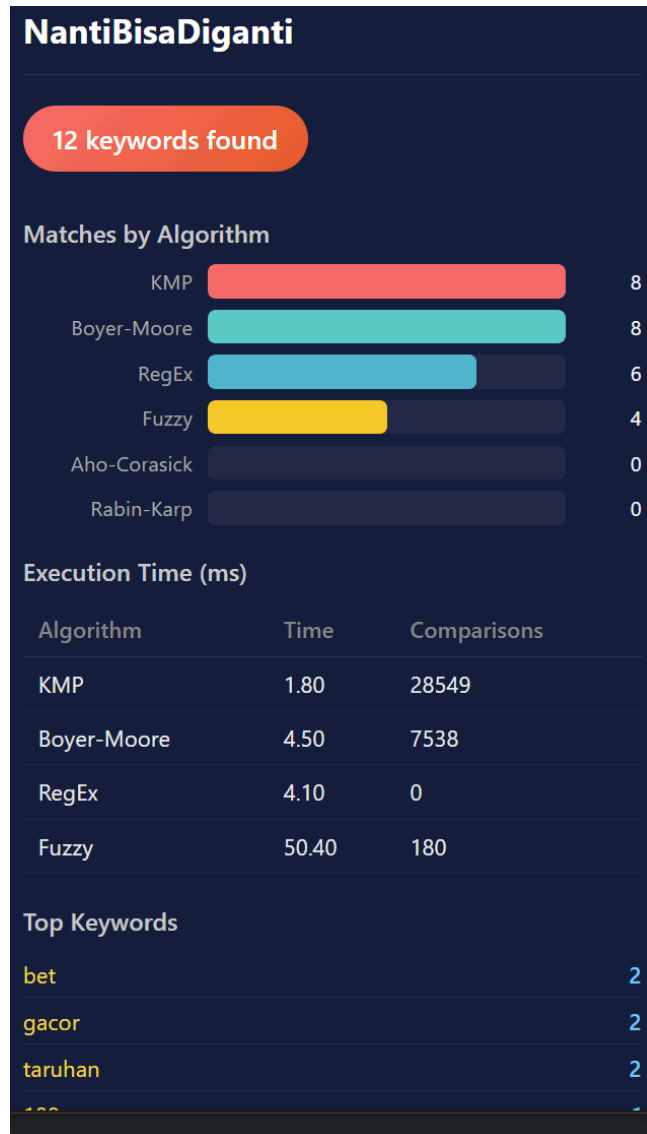
Berikut merupakan beberapa pengujian dari implementasi source code.

a.

1. Kasus 1

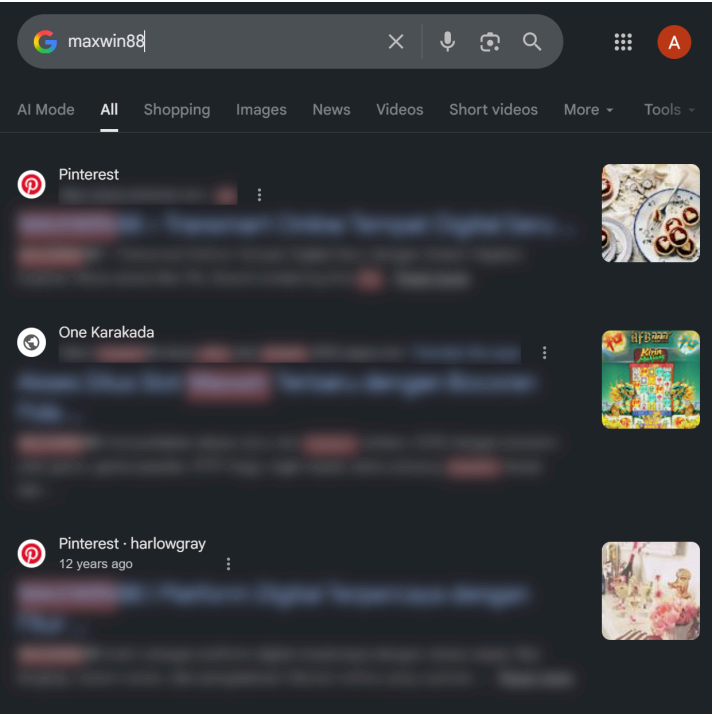
Pengujian kasus deteksi pola <keyword> dengan blur





2. Kasus 2

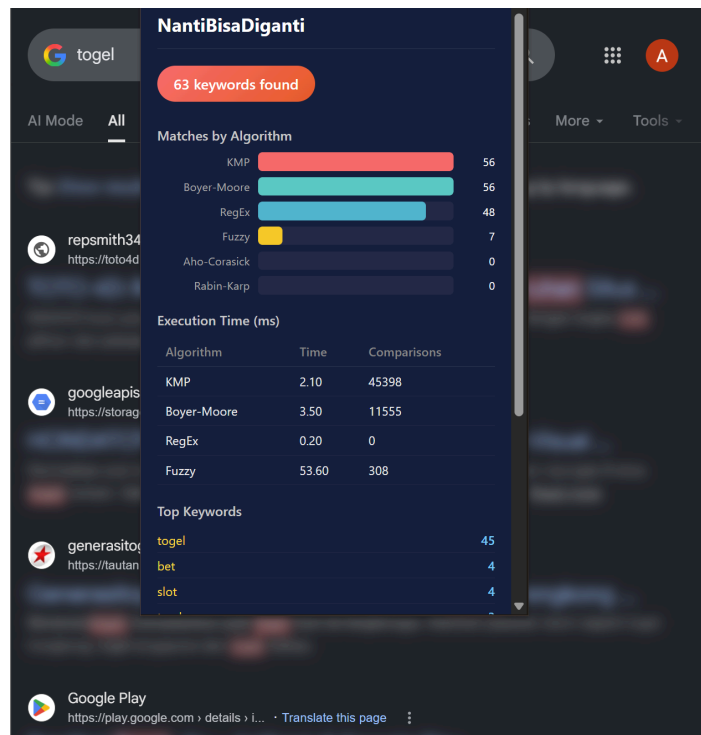
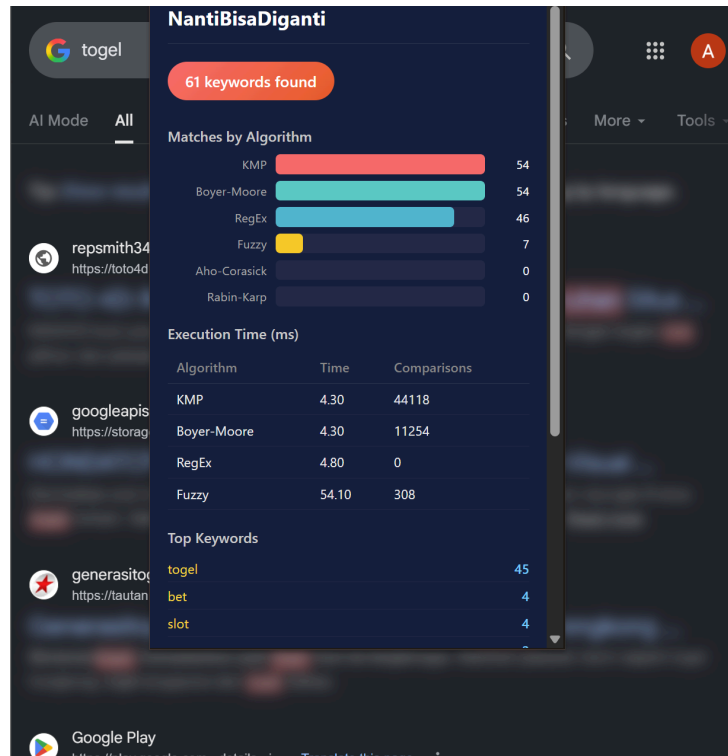
Pengujian kasus deteksi pola <keyword><angka> dengan blur





3. Kasus 3

Pengujian kasus deteksi pola <keyword> tanpa dan dengan ocr



4. Kasus 4

Waktu eksekusi dan perbandingan pengujian kasus deteksi pola
<keyword> tanpa dan dengan Aho-Corasick

Execution Time (ms)		
Algorithm	Time	Comparisons
KMP	2.70	45496
Boyer-Moore	3.00	11590
RegEx	0.20	0
Fuzzy	64.90	308
Top Keywords		
togel		45
bet		4
slot		4
taruhan		3
cuan		2
188		1
casino		1
hoki		1
judi		1
master		1
Scan		
X Clear		
☒ Blur ☐ OCR ☐ AC ☐ RK		

Execution Time (ms)		
Algorithm	Time	Comparisons
KMP	3.00	45352
Boyer-Moore	3.10	11556
RegEx	0.30	0
Fuzzy	49.20	308
Aho-Corasick	0.20	389028

Top Keywords	
togel	45
bet	4
slot	4
taruhan	3
cuan	2
188	1
casino	1
hoki	1
judi	1
master	1

Scan
✕ Clear

5. Kasus 5

Waktu eksekusi dan perbandingan pengujian kasus deteksi pola
 <keyword> tanpa dan dengan Rabin-KaRabin-Karp

Execution Time (ms)

Algorithm	Time	Comparisons
KMP	2.70	45496
Boyer-Moore	3.00	11590
RegEx	0.20	0
Fuzzy	64.90	308

Top Keywords

togel	45
bet	4
slot	4
taruhan	3
cuan	2
188	1
casino	1
hoki	1
judi	1
master	1

Scan

✕ Clear

☒ Blur ☐ OCR ☐ AC ☐ RK

Execution Time (ms)

Algorithm	Time	Comparisons
KMP	2.80	45496
Boyer-Moore	3.00	11590
RegEx	0.30	0
Fuzzy	58.70	308
Rabin-Karp	19.10	44310

Top Keywords

togel	45
bet	4
slot	4
taruhan	3
cuan	2
188	1
casino	1
hoki	1
judi	1
master	1

Scan

✕ Clear

☒ Blur ☐ OCR ☐ AC ☒ RK

4.3. Analisis Algoritma

a. Exact Matching

Algoritma	Jenis	Kompleksitas Waktu	Kompleksitas Ruang	Kelebihan	Kelemahan
KMP	Single pattern	$O(n + m)$	$O(m)$	Optimal untuk teks pendek-sedang, tidak mundur dalam teks	Hanya satu pola, preprocessing cukup
Boyer-Moore	Single pattern	$O(n/m)$ best case, $O(n*m)$ worst case	$O(m + \text{alphabet})$	Cepat untuk teks panjang & alfabet besar	Preprocessing last occurrence & good suffix
Rabin-Karp	Single/multi pattern	$O(n + m)$ rata-rata, $O(n*m)$ worst	$O(1)$	Cocok untuk multiple pattern dengan hash	Banyak false positive (collision hash)
Aho-Corasick	Multi pattern	$O(n + \text{total length patterns})$	$O(\text{total nodes})$	Paling efisien untuk banyak pola sekaligus	Konsumsi memori besar untuk trie + failure link

b. Fuzzy Matching: Untuk mendeteksi kata/karakter yang mirip secara visual

- Bobot substitusi: Identik(0), Mirip visual(0.1-0.3), Lainnya(1.0)
- Ambang batas default: 1.5
- Batasan: Perbedaan panjang maksimal 2 karakter, dan minimal kata 3 karakter
- Kompleksitas: $O(m \times n)$ per pasang kata

c. Normalisasi Teks: Untuk menyeragamkan karakter yang mirip (unicode, aksen, huruf asing) ke bentuk ASCII dasar

d. Regex Matching: Mencari pola seperti slot 88, 88 slot, slot88, dst

Bab 5

Kesimpulan dan Saran

5.1. Kesimpulan

Berdasarkan hasil implementasi dan pengujian browser extension untuk deteksi konten judi online menggunakan teknik pattern matching, dapat ditarik beberapa kesimpulan sebagai berikut

1. Algoritma KMP, Boyer-Moore, dan levenshtein berhasil diimplementasikan dengan benar dan efisien. Kedua algoritma menghasilkan output yang identik pada semua kasus pengujian, membuktikan implementasi yang konsisten, KMP dengan kompleksitas $O(n + m)$ memberikan performa linier yang stabil, sedangkan Boyer-Moore dengan rata-rata sublinier $O(n/m)$ unggul pada teks panjang dengan pola yang panjang.
2. Pendekatan berlapis terbukti efektif. Hierarki deteksi yang terdiri dari exact matching $\rightarrow$ regex matching $\rightarrow$ fuzzy matching mampu menangkap berbagai variasi konten judi online, termasuk kata kunci langsung, kata kunci dengan angka, maupun kata kunci yang di obfuskasi
3. Weighted Levenshtein Distance dengan pembobotan karakter visual terbukti tepat sasaran. Dengan memberikan bobot substitusi yang lebih kecil untuk pasangan karakter yang mirip secara visual (misalnya $O \rightarrow 0 = 0.1$, $I \rightarrow 1 = 0.1$). Sistem dapat mengenali teknik leet speak dan homoglyph yang umum digunakan oleh operator situs judi online untuk menghindari filter biasa. Threshold 1.5 memberikan keseimbangan yang baik antara sensitivitas dan spesifikasi deteksi.
4. Implementasi case-insensitive secara konsisten meningkatkan akurasi deteksi. Normalisasi teks ke huruf kapital, sebelum pencocokan memastikan bahwa variasi penulisan, seperti “gacor”, “GACOR”, ‘Gacor’, dan “GaCor” semuanya terdeteksi tanpa perlu mendaftarkan setiap variasinya secara terpisah
5. Sistem berhasil menangani kasus edge dengan baik. Pengujian menunjukkan bahwa sistem telah menangani kondisi-kondisi khusus dengan benar, antara lain: teks kosong, pattern lebih panjang dari teks, keyword satu karakter, teks yang

panjang (26.000+ karakter), serta karakter unicode fullwidth yang tidak seharusnya terdeteksi sebagai positif

5.2. Saran

Berdasarkan hasil implementasi dan analisis yang telah dilakukan, terdapat beberapa saran untuk pengembangan lebih lanjut :

1. Optimasi fuzzy matching dengan indexing atau trie. Proses fuzzy matching saat ini memiliki kompleksitas $O(W \times K \times L^2)$ di mana W adalah jumlah kata dalam teks, K jumlah keyword, dan L panjang kata. Penggunaan struktur data seperti iBK-Tree dapat mempercepat pencarian fuzzy match menjadi jauh lebih efisien untuk dataset keyword yang besar
2. Penerapan machine learning sebagai lapisan deteksi tambahan. Model klasifikasi berbasis teks dapat dilatih pada dataset halaman web judi online untuk mendeteksi konten judol berdasarkan konteks, bukan hanya kehadiran kata kunci tertentu. Ini akan membantu mengurangi false negative pada halaman yang menggunakan parafrase atau bahasa tidak langsung

Lampiran

Github Repository: https://github.com/SpicyCoinCracker/Tubes3_NantiBisaDiganti

Pernyataan Karya Orisinalitas

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Fazri Arrashyi Putra



Muh. Hartawan Haidir



Ahmad Rinofarus Mochtar

Tabel Penyelesaian

No	Poin	Ya	Tidak
1	Extension berhasil di-build dan di-load tanpa kesalahan pada chromium browser dan dikembangkan dengan TypeScript	✓	
2	KMP dan Boyer-Moore diimplementasikan from scratch	✓	
3	Regex handle format <kata><angka> dan berbagai edge case	✓	
4	Pencarian KMP & BM membaca keyword.txt secara iteratif dan tidak menggunakan built-in search function atau library eksternal	✓	
5	Exact matching dan fuzzy matching berjalan benar	✓	
6	Elemen DOM terdeteksi diberi highlight dan terhapus saat rescanning	✓	
7	Tooltip muncul saat hover dengan informasi keyword, algoritma, kemunculan, dan waktu eksekusi	✓	
8	Popup menampilkan statistik realtime (total keyword, perbandingan, waktu eksekusi, jumlah match)	✓	
9	[Bonus] Membuat Video		✓
10	[Bonus] Implementasi Algoritma Aho-Corasick dan Rabin Karp	✓	
11	[Bonus] Implementasi Censorship / Blur Teks	✓	
12	[Bonus] Implementasi Optical Character Recognition pada Gambar	✓	

Tabel Pembagian Tugas

No	Nama	Poin
1	Fazri Arrashyi Putra	Membuat bagian frontend popup, tooltip serta membantu mengerjakan algoritma bonus dan laporan.
2	Ahmad Rinofaros Muchtar	Membuat bagian content, blur, context, highlighter keywordLoader, ocr, dan scanner. Serta laporan
3	Muh. Hartawan Haidir	Mengerjakan algoritma kmp, boyer-Moore, dan levenshtein. Serta membantu mengerjakan laporan

Daftar Pustaka

T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: The MIT Press, 2022.

GeeksforGeeks, “Boyer Moore Algorithm for Pattern Searching in Python.”
<https://www.geeksforgeeks.org/dsa/boyer-moore-algorithm-in-python/> (accessed May 30, 2026).

S. Sharma, “The Boyer-Moore String Search Algorithm,” Medium.
<https://medium.com/@siddharth.21/the-boyer-moore-string-search-algorithm-674906cab162>
(accessed May 30, 2026).

A. S. Lubis, M. R. Lubis, and A. R. Hasibuan, “Analisa Perbandingan Boyer Moore dan Knuth Morris Pratt dalam Pencarian String,” [Online]. Available:
<https://media.neliti.com/media/publications/579867-analisa-perbandingan-boyer-moore-dan-knu-a9d5b9d4.pdf> (accessed May 30, 2026).